ネットアップの Apache Spark 向けストレージソリューション

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/ja-jp/netapp-solutions/data-analytics/apache-spark-solution-overview.html on March 19, 2024. Always check docs.netapp.com for the latest.

目次

ネットアップの Apache Spark 向けストレージソリューション . . . 1
TR-4570：『NetApp Storage Solutions for Apache Spark：Architecture、Use Cases、Performance Results』 . . . 1
対象読者 . . . 6
解決策テクノロジ . . . 7
NetApp Sparkソリューションの概要 . . . 9
ユースケースの概要 . . . 11
AI、ML、DLの主なユースケースとアーキテクチャ . . . 13
テスト結果 . . . 18
Hybrid Cloud解決策 の略 . . . 29
主なユースケースごとにPythonスクリプトを使用できます . . . 30
まとめ . . . 49
追加情報の参照先 . . . 49

ネットアップの Apache Spark 向けストレージソリューション

TR-4570：『NetApp Storage Solutions for Apache Spark：Architecture、Use Cases、Performance Results』

ネットアップKarthikeyan Nagalingam、Rick Huang氏

本ドキュメントでは、Apache Sparkのアーキテクチャ、お客様のユースケース、ビッグデータ分析と人工知能（AI）に関連するネットアップストレージポートフォリオについて説明します。また、一般的なHadoopシステムに対して業界標準のAI、機械学習（ML）、ディープラーニング（DL）ツールを使用してさまざまなテスト結果を提示することで、適切なSpark解決策 を選択できます。まず、Sparkアーキテクチャ、適切なコンポーネント、2つの配置モード（クラスタとクライアント）が必要です。

このドキュメントでは、構成の問題に対処するためのユースケースも紹介しています。また、Sparkでのビッグデータ分析、AI、ML、DLに関連するネットアップのストレージポートフォリオの概要についても説明しています。その後、Spark固有のユースケースとNetApp Sparkの解決策 ポートフォリオから得られたテスト結果をご紹介します。

お客様の課題

このセクションでは、小売、デジタルマーケティング、銀行業務、ディスクリート製造、プロセス製造などのデータ成長業界におけるビッグデータ分析とAI / ML / DLに関するお客様の課題に焦点を当てます。 政府機関やプロフェッショナルサービスも利用できます。

予測不可能なパフォーマンス

従来型のHadoop導入では、一般にコモディティハードウェアを使用します。パフォーマンスを向上させるには、ネットワーク、オペレーティングシステム、Hadoopクラスタ、Sparkなどのエコシステムコンポーネント、ハードウェアをチューニングする必要があります。それぞれのレイヤを調整しても、Hadoopは環境内のハイパフォーマンスを想定して設計されていないコモディティハードウェア上で実行されるため、必要なパフォーマンスレベルを達成することは困難です。

メディアおよびノードの障害

通常の状態でも、コモディティハードウェアは障害が発生しやすくなります。データノードの1つのディスクに障害が発生すると、Hadoopマスターはデフォルトでそのノードを正常な状態ではないとみなします。次に、レプリカから正常なノードに、そのノードからネットワーク経由で特定のデータをコピーします。このプロセスにより、Hadoopジョブのネットワークパケット速度が低下します。正常な状態に戻ったら、クラスタでデータを再度コピーしてレプリケートデータを削除する必要があります。

Hadoopベンダーロックイン

Hadoopディストリビュータは独自のバージョン管理機能を備えたHadoopディストリビューションを所有しており、これによってお客様はこれらのディストリビューションにロックされます。ただし、多くのお客様が、特定のHadoopディストリビューションにお客様を結び付けることのないインメモリ分析をサポートしている必要があります。ディストリビューションを自由に変更し、分析機能を活用する必要があります。

複数の言語をサポートしていない

MapReduce Javaプログラムに加えて、ジョブを実行するために複数の言語のサポートが必要になることがよくあります。SQLやスクリプトなどのオプションを使用すると、回答を取得する柔軟性が高まり、データを整理して取得するためのオプションが増え、データを分析フレームワークにすばやく移動できます。

使用の難しさ

しばらくの間、Hadoopが使いにくいとユーザからクレームを受けています。Hadoopは新しいバージョンが登場するたびにシンプルかつ強力になりましたが、今もこの批評家は存続しています。Hadoopを使用するには、JavaおよびMapReduceのプログラミングパターンを理解する必要があります。これは、データベース管理者や、従来のスクリプトスキルセットを持つユーザにとっての課題です。

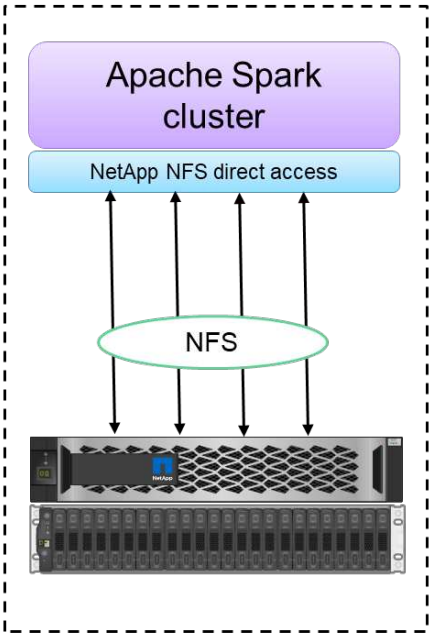
複雑なフレームワークとツール

企業のAIチームは、さまざまな課題に直面します。導入エコシステムやアプリケーションごとに専門的なデータサイエンスの知識があるとしても、ツールやフレームワークは単に相互に変換されるわけではありません。データサイエンスプラットフォームは、Spark上に構築された対応するビッグデータプラットフォームとシームレスに統合する必要があります。データ移動が容易で、再利用可能なモデル、すぐに使えるコード、プロトタイプ作成、検証、バージョン管理、共有、再利用といったベストプラクティスに対応するツールを備えています。 また、モデルを迅速に本番環境に導入できます。

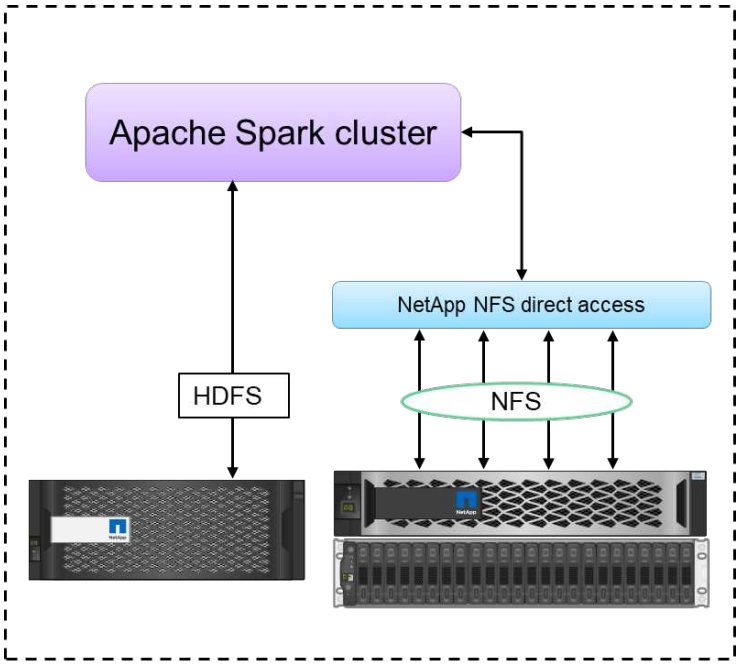
ネットアップを選ぶ理由

ネットアップでは、次の方法でSpark体験を向上させています。

- NetApp NFSから直接アクセスする（次の図を参照）ことで、データを移動したりコピーしたりすることなく、既存または新規のNFSv3 / NFSv4データに対してビッグデータ分析ジョブを実行できます。データの複数のコピーが作成されるため、ソースとデータを同期する必要がありません。
- ストレージ効率が向上し、サーバのレプリケーションが不要になります。たとえば、NetApp EシリーズHadoop解決策 にはデータのレプリカが3つではなく2つ必要であり、FAS Hadoop解決策 にはデータソースが必要ですが、データのレプリケーションやコピーは必要ありません。ネットアップのストレージソリューションは、サーバ間のトラフィックも削減します。
- ドライブおよびノードの障害時のHadoopジョブとクラスタの動作が向上します。
- データ取り込みのパフォーマンスが向上します。



Configuration 1: NFS as primary storage



Configuration 2: HDFS and NFS in single Spark cluster

たとえば、金融機関や医療機関では、ある場所から別の場所へデータを移動する際に法的義務を果たす必要がありますが、これは容易な作業ではありません。このシナリオでは、NetApp NFSから直接アクセスして、財務データと医療データを元の場所から分析します。もう1つの主なメリットは、NetApp NFS直接アクセスを使用することで、ネイティブのHadoopコマンドを使用してHadoopデータを簡単に保護できるようになることと、ネットアップの充実したデータ管理ポートフォリオでデータ保護ワークフローを実現できることです。

NetApp NFS直接アクセスでは、HadoopクラスタとSparkクラスタに対して次の2種類の導入オプションを提供しています。

- デフォルトでは、HadoopクラスタまたはSparkクラスタは、データストレージとデフォルトのファイルシステムにHadoop Distributed File System（HDFS；Hadoop分散ファイルシステム）を使用しています。NetApp NFSから直接アクセスできるため、デフォルトのHDFSをデフォルトのファイルシステムとしてNFSストレージに置き換えることができ、NFSデータを直接分析できます。
- もう1つの導入オプションでは、NetApp NFS直接アクセスを使用して、1つのHadoopクラスタまたはSparkクラスタ内でHDFSを追加のストレージとして構成することもできます。この場合、 NFS エクスポートを介してデータを共有し、 HDFS データと同じクラスタからデータにアクセスできます。

NetApp NFS直接アクセスを使用する主な利点は次のとおりです。

- 現在の場所からデータを分析することで、分析データをHDFSなどのHadoopインフラに移動するための時間とパフォーマンスのかかる作業を回避できます。
- レプリカの数を3分の1から1に削減。
- ユーザを分離してコンピューティングとストレージを分離し、個別に拡張
- ONTAP の充実したデータ管理機能を活用して、エンタープライズデータを保護
- Hortonworksデータプラットフォームの認定。
- ハイブリッドデータ分析の導入を実現
- 動的なマルチスレッド機能を活用してバックアップ時間を短縮

を参照してください "TR-4657 ：ネットアップのハイブリッドクラウドデータソリューション - Spark と Hadoop はお客様のユースケースに基づいています" Hadoopデータのバックアップ、クラウドからオンプレミスへのバックアップ、ディザスタリカバリ、既存のHadoopデータに対するDevTestの有効化、データ保護とマルチクラウド接続の実現、分析ワークロードの高速化を実現します。

次のセクションでは、Sparkのお客様にとって重要なストレージ機能について説明します。

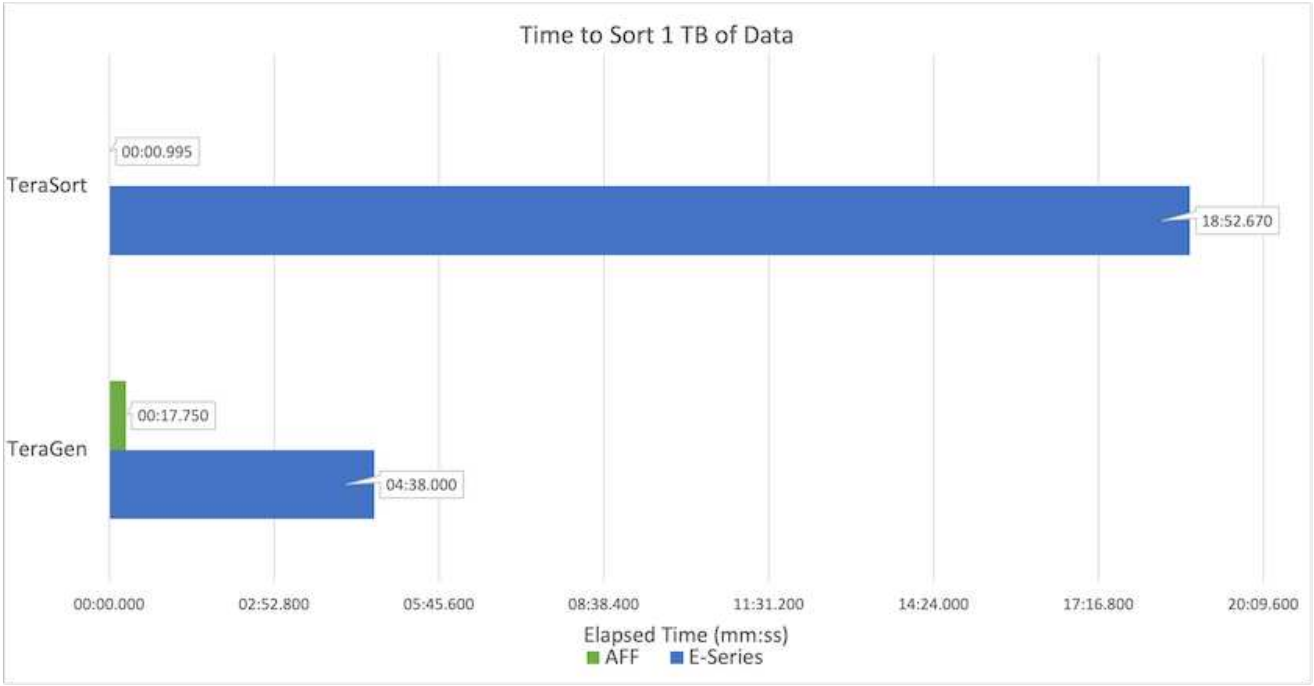
ストレージ階層化

Hadoopストレージ階層化を使用すると、ストレージポリシーに従ってさまざまなタイプのファイルを格納できます。ストレージ・タイプには' hot "cold "warm "all_sssd "one _sssd 'が含まれます 「lazy_persist`」。

<<<<そこで、NetApp AFF ストレージコントローラと、ストレージポリシーが異なるSSDとSASドライブを搭載したEシリーズストレージコントローラでHadoopストレージの階層化を検証しました。AFF-A800のSparkクラスタには4つのコンピュートワーカーノードがあり、Eシリーズのクラスタには8つのノードがあります。主な用途は、ソリッドステートドライブ（SSD）とハードドライブディスク（HDD）のパフォーマンスを比較することです。

ネットアップのAFF ストレージコントローラと、ストレージポリシーが異なるSSDおよびSASドライブを搭載したEシリーズストレージコントローラでHadoopストレージの階層化を検証しました。AFF-A800のSparkクラスタには4つのコンピュートワーカーノードがあり、Eシリーズのクラスタには8つのノードがあります。これは主に、ソリッドステートドライブとハードドライブディスクのパフォーマンスを比較するために行いました。>>>>>>>>a51c9ddf73ca69e1120ce05edc7b0b9607b96eae

次の図に、ネットアップのHadoop SSD向けソリューションのパフォーマンスを示します。



- ベースラインNL-SAS構成では、8つのコンピューティングノードと96本のNL-SASドライブを使用しました。この構成では、1TBのデータが4分38秒で生成され、を参照してください "TR-3969『NetApp Eシリーズ解決策 for Hadoop』" クラスタとストレージ構成の詳細については、を参照してください。
- TeraGenを使用した場合、SSD構成ではNL-SAS構成よりも1TBのデータが15.66x高速で生成されました。さらに、SSD構成では、コンピューティングノードの半分とディスクドライブの半分（合計24本のSSDドライブ）が使用されていました。ジョブの完了時間に基づき、NL-SAS構成の約2倍の速さで処理されました。
- SSD構成は、TeraSortを使用してNL-SAS構成の1TBのデータを1138.36倍高速にソートしました。さらに、SSD構成では、コンピューティングノードの半分とディスクドライブの半分（合計24本のSSDドライブ）が使用されていました。そのため、ドライブあたりの速度は、NL-SAS構成の約3倍です。< < < < < < < < < < < < < < 頭部
- ここで重要なのは、回転式ディスクからオールフラッシュに移行することでパフォーマンスを向上させることです。コンピューティングノードの数がボトルネックになっていません。ネットアップのオールフラッシュストレージなら、ランタイムのパフォーマンスを大幅に向上できます。
- NFSでは、データがすべてプールされる機能と同等で、ワークロードに応じてコンピューティングノードの数を減らすことができました。コンピューティングノードの数を変更した場合、Apache Sparkクラスタユーザは手動でデータをリバランシングする必要がありません。

- つまり、回転式ディスクからオールフラッシュに移行することでパフォーマンスが向上します。コンピューティングノードの数がボトルネックになっていません。ネットアップのオールフラッシュストレージな

ら、ランタイムのパフォーマンスを大幅に向上できます。

- NFSでは、データがすべてプールされる機能と同等で、ワークロードに応じてコンピューティングノードの数を減らすことができました。コンピューティングノードの数を変更した場合、Apache Sparkクラスタユーザは手動でデータをリバランシングする必要がありません。>>>>>>>>a51c9ddf73ca69e1120ce05edc7b0b9607b96eae

パフォーマンスの拡張-スケールアウト

AFF 解決策 のHadoopクラスタの処理能力を強化する必要がある場合は、適切な数のストレージコントローラを使用してデータノードを追加できます。ワークロードの特性に応じて、ストレージコントローラアレイごとにデータノードを4つから始めて、ストレージコントローラごとにデータノードを8つに増やすことを推奨します。

AFF とFAS はインプレース分析に最適です。コンピューティング要件に基づいて、ノードマネージャを追加できます。また、ノンストップオペレーション機能により、ダウンタイムなしでストレージコントローラをオンデマンドで追加できます。AFF とFAS を備えた豊富な機能を備えており、NVMeメディアのサポート、効率性の保証、データ削減、QoS、予測分析、 クラウドの階層化、レプリケーション、クラウドの導入、セキュリティお客様が要件を満たせるように、ネットアップでは、追加のライセンスコストなしでファイルシステム分析、クォータ、オンボックスロードバランシングなどの機能を提供しています。ネットアップは、同時ジョブ数やレイテンシの低減、処理の簡易化、1秒あたりのスループットの向上といった、競合他社よりも優れたパフォーマンスを提供しています。さらに、ネットアップのCloud Volumes ONTAP は、主要な3つのクラウドプロバイダすべてで動作します。

パフォーマンスの拡張-スケールアップ

ストレージ容量を追加する必要がある場合は、スケールアップ機能を使用して、AFF 、FAS 、およびEシリーズシステムにディスクドライブを追加できます。Cloud Volumes ONTAP を使用してストレージをPBレベルに拡張するには、使用頻度の低いデータをブロックストレージからオブジェクトストレージに階層化し、追加のコンピューティングなしでCloud Volumes ONTAP ライセンスをスタックするという2つの要素を組み合わせます。

複数のプロトコル

ネットアップシステムは、SAS、iSCSI、FCP、InfiniBandなど、Hadoop導入のほとんどのプロトコルをサポートしています。 およびNFSが必要です。

運用およびサポートされるソリューション

本ドキュメントに記載されているHadoopソリューションは、ネットアップによってサポートされています。これらのソリューションは、主要なHadoopディストリビュータでも認定されています。詳細については、を参照してください "MapR" サイト、 "Hortonworks" サイト、Cloudera "認定資格" および "パートナー" サイト：

対象読者

分析とデータサイエンスの世界は、ITとビジネスのさまざまな分野に影響を与えています。

- データサイエンティストは、選択したツールとライブラリを柔軟に使用できる必要があります。
- データエンジニアは、データの流れと配置場所を把握する必要があります。

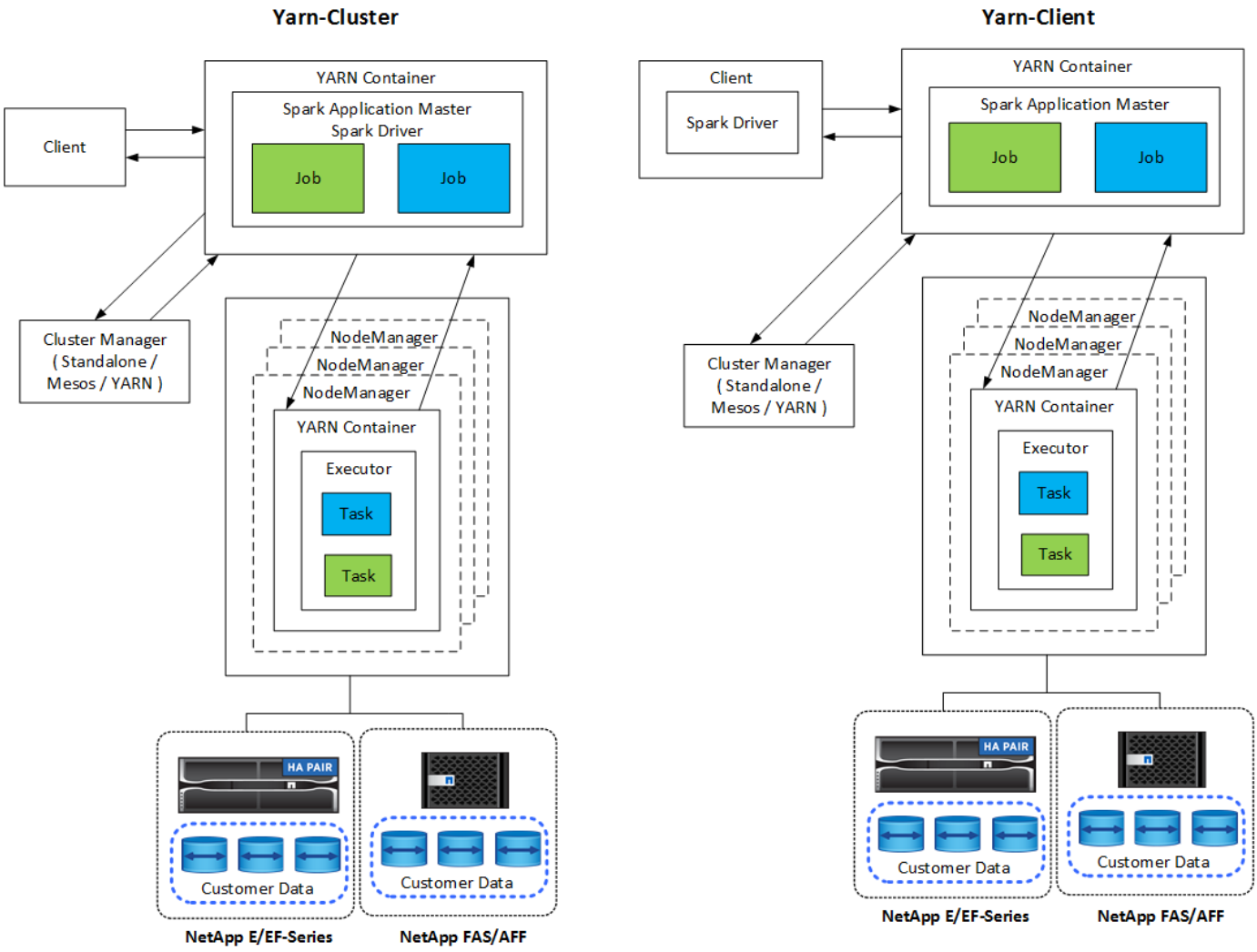
- DevOpsエンジニアは、新しいAIアプリケーションやMLアプリケーションをCIパイプラインやCDパイプラインに統合するためのツールを必要としています。
- クラウド管理者とアーキテクトは、ハイブリッドクラウドリソースを設定し、管理できる必要があります。
- ビジネスユーザは、分析、AI、ML、DLアプリケーションにアクセスしたいと考えています。

このテクニカルレポートでは、NetApp AFF 、Eシリーズ、StorageGRID 、NFSへの直接アクセス、Apache Spark HorovodとKerasは、これらの役割のそれぞれがビジネスに価値をもたらすのを支援します。

解決策テクノロジ

Apache Sparkは、Hadoop Distributed File System（HDFS）と直接連携するHadoopアプリケーションを作成するための一般的なプログラミングフレームワークです。Sparkは本番環境に対応しており、ストリーミングデータの処理をサポートしています。MapReduceよりも高速です。Sparkには、効率的なイテレーションのためのメモリ内データキャッシングが設定可能であり、Sparkシェルはデータの学習と探索のためにインタラクティブです。Sparkでは、Python、Scala、Javaでアプリケーションを作成できます。Sparkアプリケーションは、1つ以上のタスクを持つ1つ以上のジョブで構成されています。

すべてのSparkアプリケーションにはSparkドライバがあります。yarn -クライアントモードでは、ドライバはクライアント上でローカルに実行されます。Yarn -クラスタモードでは、ドライバはアプリケーションマスター上のクラスタで実行されます。クラスタモードでは、クライアントが切断してもアプリケーションは引き続き実行されます。



クラスタマネージャは3種類あります。

- *スタンドアロン。*このマネージャーはSparkの一部であり、クラスタのセットアップが簡単です。
- * Apache Mesos.* MapReduceなどのアプリケーションも実行される一般的なクラスタマネージャです。
- * Hadoop糸。*これはHadoop 3のリソースマネージャーです。

レジリエントな分散データセット（RDD）はSparkの主要コンポーネントです。RDDは、クラスタ内のメモリに格納されているデータから、失われたデータや欠落しているデータを再作成し、ファイルから取得した初期データまたはプログラムによって作成された初期データを格納します。RDDは、ファイル、メモリ内のデータ、または別のRDDから作成されます。Sparkプログラミングは、変換とアクションという2つの操作を実行します。既存のRDDに基づいて新しいRDDが作成されます。アクションはRDDから値を返します。

変換とアクションはSparkのデータセットとDataFramesにも適用されます。データセットとは、分散されたデータの集合であり、Spark SQLの最適化された実行エンジンの利点とともに、RDDの利点（強力なタイピング、ラムダ関数の使用）を提供します。データセットはJVMオブジェクトから作成し、機能変換（マップ、フラットマップ、フィルタなど）を使用して操作できます。DataFrameは、名前付き列に編成されたデータセットです。概念的には、リレーショナルデータベースのテーブルまたはR/Pythonのデータフレームに相当します。データフレームは、構造化データファイル、HiveまたはHBaseのテーブル、オンプレミスまたはクラウドの外部データベース、既存のRDDなど、さまざまなソースから構築できます。

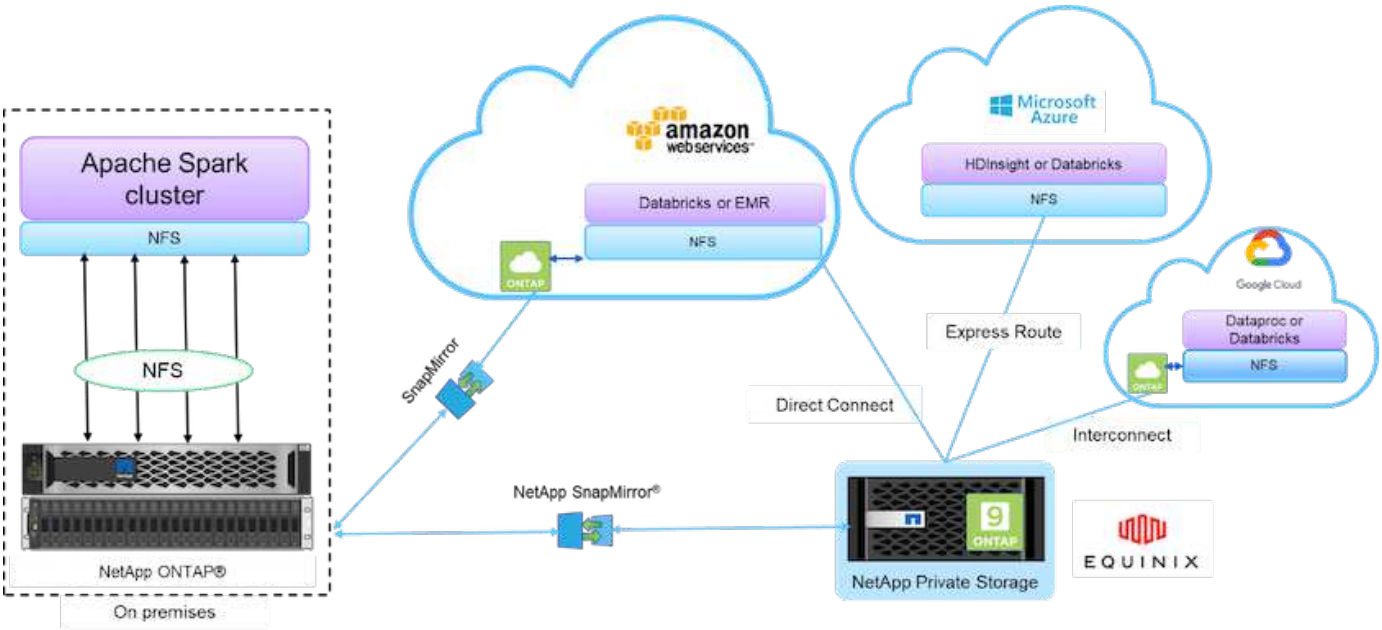
Sparkアプリケーションには1つ以上のSparkジョブが含まれています。ジョブは実行者でタスクを実行し、実

行者はYarnコンテナで実行します。各実行者は1つのコンテナで実行され、実行者はアプリケーションのライフサイクルを通して存在します。実行者はアプリケーションの起動後に修正され、yarnは割り当て済みのコンテナのサイズを変更しません。実行者は、メモリ内のデータに対してタスクを同時に実行できます。

NetApp Sparkソリューションの概要

ネットアップには、FAS / AFF、Eシリーズ、Cloud Volumes ONTAP の3つのストレージポートフォリオがあります。AFF とONTAP を搭載したEシリーズは、Apache Spark搭載のHadoopソリューション向けに検証済みです。

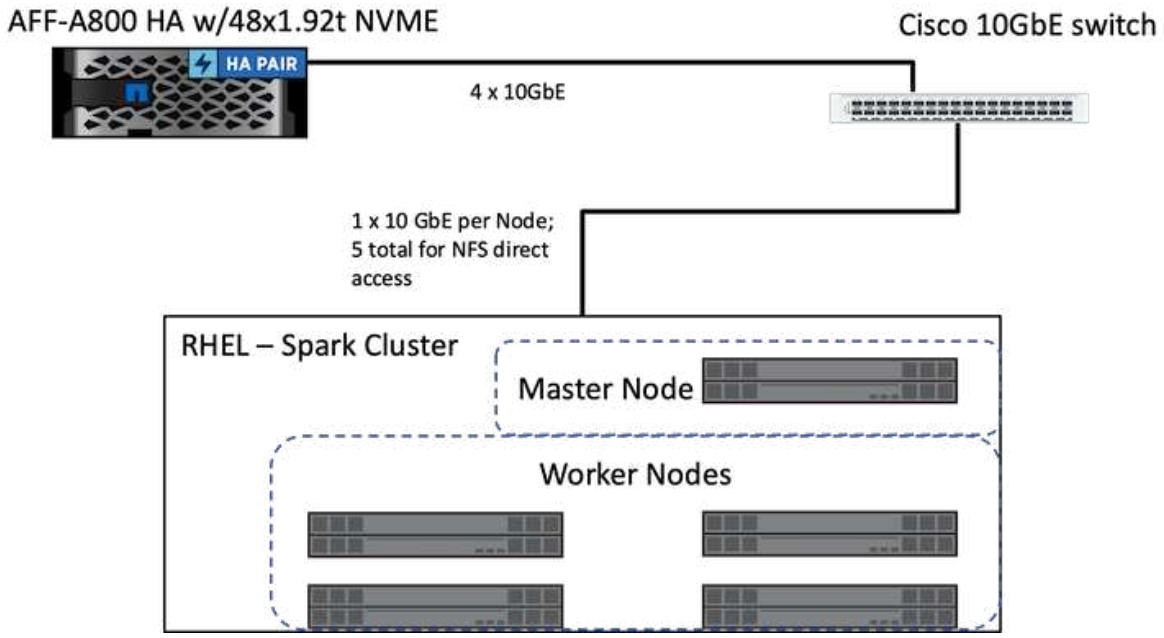
ネットアップのデータファブリックは、以下の図に示すように、データアクセス、制御、保護、セキュリティのためのデータ管理サービスとアプリケーション（ビルディングブロック）を統合しています。



上の図の構成要素は次のとおりです。

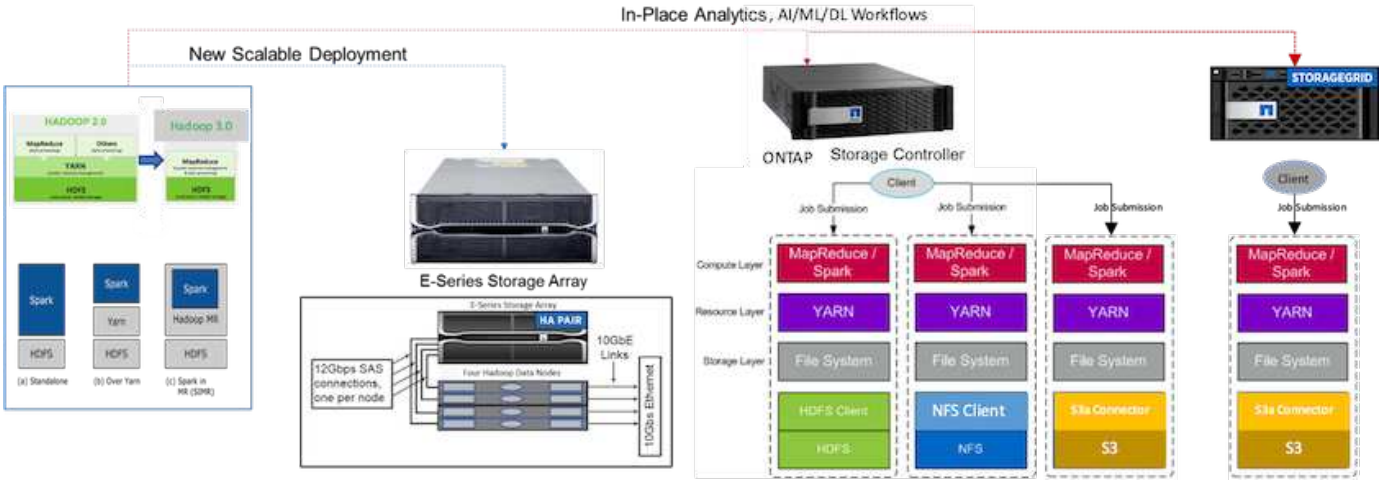
- * NetApp NFS 直接アクセス。 * 最新の Hadoop クラスタと Spark クラスタを、ソフトウェアやドライバの追加の必要なしに NetApp NFS ボリュームに直接アクセスできます。
- * ネットアップの Cloud Volumes ONTAP とクラウドボリュームサービス。 * ソフトウェア定義型の接続ストレージ。 Amazon Web Services （ AWS ）で実行されている ONTAP または Microsoft Azure クラウドサービスで実行されている Azure NetApp Files （ ANF ）に基づいています。
- * NetApp SnapMirrorテクノロジ。*オンプレミスとONTAP クラウドまたはNPSインスタンス間のデータ保護機能を提供します。
- * クラウド・サービス・プロバイダー。 * これらのプロバイダーには、 AWS 、 Microsoft Azure 、 Google Cloud 、 IBM Cloud が含まれます。
- * PaaS * AWS の Amazon Elastic MapReduce （ EMR ）や Databricks 、 Microsoft Azure HDInsight 、 Azure Databricks などのクラウドベースの分析サービスを利用できます。

次の図は、Sparkの解決策 とネットアップストレージを示しています。

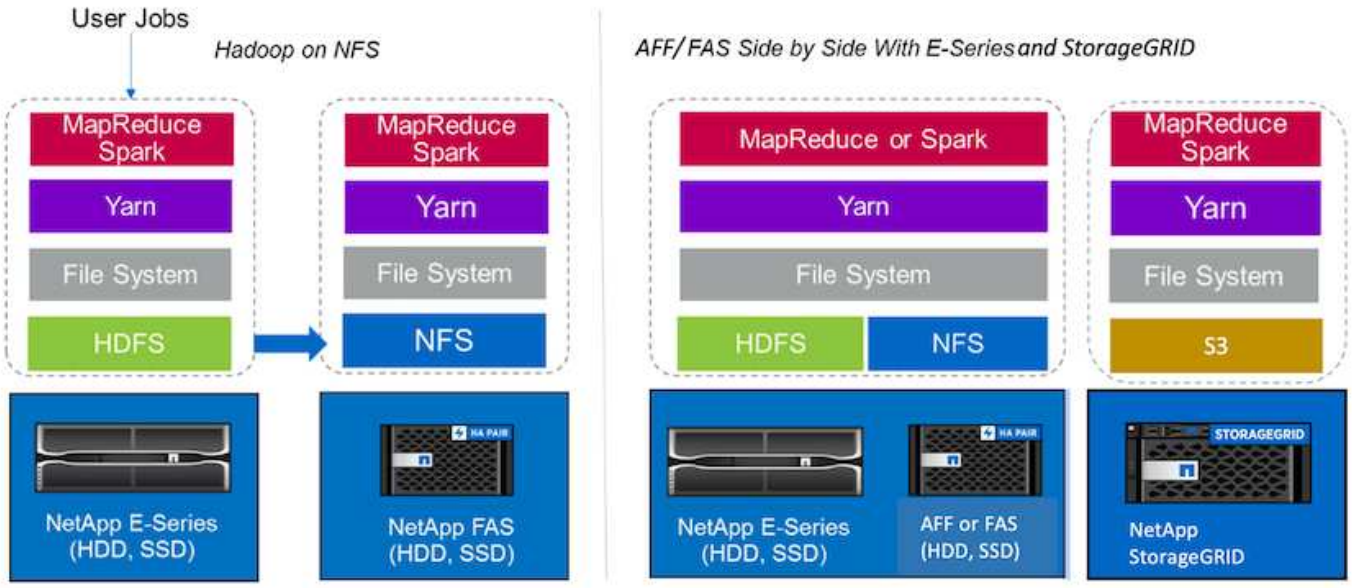


ONTAP Spark解決策 は、既存の本番データへのアクセスを使用して、インプレース分析、AI、ML、DLのワークフローに、ネットアップNFSダイレクトアクセスプロトコルを使用しています。Hadoopノードで使用可能な本番データは、インプレース分析ジョブ、AIジョブ、MLジョブ、DLジョブを実行するためにエクスポートされます。データにアクセスしてHadoopノード内で処理することができ、NetApp NFSに直接アクセスするかどうかは関係ありません。Sparkでは、スタンドアロンのクラスタマネージャまたは「yarn」クラスタマネージャを使用して、「」を使用してNFSボリュームを構成できます<file:///<target_volume>`。3つのユースケースに異なるデータセットを使用して検証しました。これらの検証の詳細については、「テスト結果」セクションを参照してください。 （XRef）

次の図は、NetApp Apache Spark / Hadoopストレージの位置付けを示しています。



また、EシリーズSparkの解決策 、AFF / FAS ONTAP Spark解決策 、StorageGRID Spark解決策 の独自の機能を特定し、詳細な検証とテストを実施しました。ネットアップでは、今回の調査結果に基づいてStorageGRIDて、新規導入時と拡張性に優れた新規導入時にEシリーズ解決策 を使用し、既存のNFSデータを使用したインプレース分析、AI、ML、DL、DLのワークロードにはAFF / FAS解決策 を、オブジェクトストレージが必要な場合には最新のデータ分析に使用することを推奨しています。



データレイクは、分析、AI、ML、DLの各ジョブに使用できる、ネイティブ形式の大規模データセット用のストレージリポジトリです。Eシリーズ、AFF / FAS、StorageGRID SG6060 Sparkソリューション用のデータレイクリポジトリを構築しました。Eシリーズシステムでは、Hadoop SparkクラスタへのHDFSアクセスが提供されますが、既存の本番環境のデータには、NFSの直接アクセスプロトコルを通じてHadoopクラスタへアクセスされます。オブジェクトストレージに配置されるデータセットに対しては、NetApp StorageGRID によってS3とS3aのセキュアなアクセスが提供されます。

ユースケースの概要

このページでは、この解決策 を使用できるさまざまな領域について説明します。

ストリーミングデータ

Apache Sparkはストリーミングデータを処理できます。ストリーミングデータは、抽出、変換、読み込み（ETL）プロセス、データのエンリッチ化、イベント検出のトリガー、複雑なセッション分析に使用されます。

- *ストリーミングETL。*データは継続的に消去され、データストアにプッシュされる前に集約されます。Netflixでは、KafkaストリーミングとSparkストリーミングを使用して、リアルタイムのオンラインムービーの推奨事項とデータ監視解決策 を構築しています。このソリューションでは、さまざまなデータソースから1日数十億件のイベントを処理できます。ただし、従来のETLではバッチ処理の処理方法が異なります。このデータは最初に読み取られ、次にデータベースに書き込まれる前にデータベース形式に変換されます。
- データのエンリッチ化。 Sparkストリーミングは、静的データを活用してライブデータを強化し、よりリアルタイムのデータ分析を可能にします。たとえば、オンライン広告主は、顧客行動に関する情報に基づいてパーソナライズされたターゲット広告を配信できます。
- イベント検出のトリガ。 Sparkストリーミングを使用すると、重大な問題を示す可能性のある異常な動作をすばやく検出して対応することができます。たとえば、金融機関はトリガーを使用して不正取引を検出および停止し、病院はトリガーを使用して患者のバイタルサインで検出された危険な健康状態変化を検出します。
- 複雑なセッション分析。 Sparkストリーミングは、Webサイトやアプリケーションにログインした後で、ユーザーのアクティビティなどのイベントを収集し、それらをグループ化して分析します。たとえ

ば、Netflixでは、この機能を使用してリアルタイムの動画推奨を提供しています。

より多くのストリーミングデータ構成、ConFluent Kafkaの検証、およびパフォーマンステストについては、を参照してください "TR-4912 ：『 Best Practices guidelines for ConFluent Kafka Tiered Storage with NetApp 』"。

機械学習

Sparkの統合フレームワークを使用すると、機械学習ライブラリ（MLlib）を使用してデータセットに対して繰り返しクエリを実行できます。MLlibは、予測インテリジェンス、マーケティング目的での顧客セグメンテーション、感情分析など、一般的なビッグデータ機能のためのクラスタリング、分類、次元縮小などの領域で使用されます。MLlibは、ネットワークセキュリティで、悪意のあるアクティビティの兆候を示すデータパケットをリアルタイムで検査するために使用されます。セキュリティプロバイダは、新しい脅威について学び、ハッカーの一歩先を行きながら、クライアントをリアルタイムで保護することができます。

ディープラーニング

TensorFlowは、業界全体で使用されている一般的なディープラーニングフレームワークです。TensorFlowは、CPUまたはGPUクラスタでの分散トレーニングをサポートしています。このトレーニングは分散されているため、ユーザはディープレイヤを大量に含む大容量のデータを使用してトレーニングを実行できます。

最近まで、Apache SparkとTensorFlowを使用したい場合は、PySparkでTensorFlowに必要なすべてのETLを実行し、中間ストレージにデータを書き込む必要がありました。そのデータは、実際のトレーニングプロセス用にTensorFlowクラスタにロードされます。このワークフローでは、ETL用とTensorFlowの分散トレーニング用の2つの異なるクラスタを管理する必要がありました。複数のクラスタを実行して保守する作業は、一般に煩雑で時間もかかりました。

以前のSparkバージョンのデータフレームとRDDは、ランダムアクセスが制限されていたため、ディープラーニングには適していませんでした。Spark 3.0では、プロジェクト水素を使用してディープラーニングフレームワークのネイティブサポートが追加されました。このアプローチにより、Sparkクラスタで非MapReduceベースのスケジューリングが可能になります。

対話式解析

Apache Sparkは、SQL、R、Pythonを含むSpark以外の開発言語でサンプリングしなくても、探索クエリを実行するのに十分な速さです。Sparkは視覚化ツールを使用して複雑なデータを処理し、対話式に視覚化します。Spark with structured streamingは、Web分析のライブデータに対してインタラクティブなクエリを実行します。これにより、Web訪問者の現在のセッションに対してインタラクティブなクエリを実行できます。

推薦システム

推薦システムは長年にわたり、オンラインショッピング、オンラインエンターテインメント、その他多くの業界の劇的な変化に企業や消費者が対応してきたため、私たちの生活に大きな変化をもたらしてきました。実際、これらのシステムは、AIの本番運用における成功事例として最も顕著に表れています。多くの実用的なユースケースでは、推薦システムを会話型AIやチャットボットと組み合わせてNLPバックエンドに接続することで、関連情報を取得し、有益な推論を作成しています。

現在、多くの小売業者は、オンラインでの購入や店舗での集荷、縁石側での集荷、セルフチェックアウト、スキャンと外出など、新しいビジネスモデルを採用しています。これらのモデルは、COVID-19のパンデミックでショッピングの安全性と消費者の利便性を高めることで、注目を引くようになっています。このようなデジタルトレンドの拡大には、AIが欠かせません。こうしたトレンドは消費者の行動に左右され、その逆も同様です。お客様のニーズの高まりに応え、カスタマーエクスペリエンスの強化、運用効率の向上、収益の拡大を実

現するために、ネットアップは、機械学習とディープラーニングのアルゴリズムを使用して、より迅速で正確な推奨システムを設計できるよう、大企業のお客様とビジネスを支援しています。

コラボレーションフィルタリング、コンテンツベースのシステム、ディープラーニングレコメンダモデル（DLRM）、ハイブリッド手法など、推奨事項を提供するために使用される一般的な手法がいくつかあります。以前は、PySparkを使って、推奨システムを作成するためのコラボレーションフィルタリングを実装していました。Spark MLlibは、コラボレーションフィルタリングのために交互に最小二乗（ALS）を実装しています。これは、DLRMが増加する前に、企業間で非常に人気のあるアルゴリズムです。

自然言語処理

会話型AIは、自然言語処理（NLP）によって実現され、コンピュータが人間と通信するのを支援するAIのブランチです。NLPは、スマートアシスタントやチャットボット、Google検索、予測テキストなど、業界のあらゆる業種や多くのユースケースで広く使用されています。に従って "Gartner社" 予測：2022年までに、人の70%が日常的に会話型AIプラットフォームとやり取りしています。人間と機械の間で質の高い会話を行うには、応答が迅速かつインテリジェントで、自然な音声である必要があります。

お客様は、NLPモデルとAutomatic Speech Recognition（ASR）モデルを処理してトレーニングするために大量のデータを必要としています。また、エッジ、コア、クラウドにわたってデータを移動する必要もあり、推論を数ミリ秒で実行して人間との自然な通信を確立する機能も必要です。NetApp AIとApache Sparkは、コンピューティング、ストレージ、データ処理、モデルトレーニング、微調整、 そして展開。

感情分析とは、NLP内で、肯定的、否定的、または中立的な感情がテキストから抽出される研究領域のことです。感情分析には、サポートセンターの従業員のパフォーマンスを発信者との会話から適切な自動チャットボット応答まで、さまざまなユースケースがあります。また、四半期ごとの収益呼において、企業の代表者と対象者の間のやり取りに基づいて会社の株価を予測するためにも使用されています。さらに、感情分析を使用して、ブランドが提供する製品、サービス、サポートに関するお客様の見解を判断できます。

使用しました "SparkのNLPです" ライブラリ元 "ジョンスノーラボ" を含むTransformers（BERT）モデルから事前にトレーニングされたパイプラインと双方向エンコーダリプレゼンテーションをロードするため "財務ニュースのセンチメント" および "FinBert"トークン化、Named Entity Recognition、モデルトレーニング、フィッティング、センチメント分析を大規模に実施しています。Spark NLPは、BERT、Albert、Electra、XLNet、DistilBERTなどの最先端のトランスを提供する唯一のオープンソースNLPライブラリです。 Roberta、DeberTa、XLM-Roberta、Longform, Elmo ユニバーサルセンテンスエンコーダー、Google T5、MarianMT、およびGPT2。このライブラリはPythonとRだけでなく、Apache Sparkをネイティブに拡張することで、JVMエコシステム（Java、Scala、Kotlin）でも大規模に動作します。

AI、ML、DLの主なユースケースとアーキテクチャ

主なAI、ML、DLのユースケースと手法は、以下のセクションに分類できます。

SparkのNLPパイプラインとTensorFlow分散推論です

次のリストには、データサイエンスコミュニティで採用されている最も一般的なオープンソースのNLPライブラリが、さまざまな開発レベルで含まれています。

- "Natural Language Toolkit（NLTK）"。すべてのNLP手法に対応する完全なツールキットです。2000年代初頭から維持されています。
- "TextBLOB"。NLTKとPatternの上に構築された使いやすいNLPツールPython API。
- "Stanford Core NLP"。Stanford NLP Groupが開発したJavaのNLPサービスとパッケージ。

- "Gensim氏"。人間のトピックモデリングは、チェコデジタル数学ライブラリプロジェクトのPythonスクリプトの集合として開始されました。
- "スパレーシー"。PythonとCythonを使用した、トランスフォーマ用GPUアクセラレーションを備えたエンドツーエンドの産業用NLPワークフロー。
- "Fasttextの場合"。FacebookのAI Research（Fair）ラボで作成された単語埋め込みや文分類の学習用の無料の軽量オープンソースNLPライブラリです。

Spark NLPは、あらゆるNLPタスクと要件に対応する単一のユニファイド解決策 です。拡張性が高く、パフォーマンスが高く、精度の高いNLPベースのソフトウェアを、実稼働環境で使用できます。また、転移学習を活用し、研究やさまざまな業界における最新のアルゴリズムとモデルを実装しています。Sparkは上記のライブラリを完全にサポートしていないため、Spark NLPはの上に構築されました "Spark ML" Sparkの汎用インメモリ分散データ処理エンジンを、ミッションクリティカルな本番ワークフロー向けのエンタープライズクラスのNLPライブラリとして活用しよう。アノテータは、ルールベースのアルゴリズム、機械学習、TensorFlowを利用してディープラーニングの実装を強化しています。トークン化、レマタイ化、語幹化、部分読み上げタギング、名前付きエンティティー認識など、一般的なNLPタスクを取り上げますが、これらに限定されません。スペルチェックと感情分析。

トランスフォーマー（BERT）の双方向エンコーダリプレゼンテーションは、NLPのトランスベースの機械学習技術です。事前トレーニングと微調整の概念を普及させました。BERTの変圧器アーキテクチャは機械翻訳から生まれたもので、回帰型ニューラルネットワーク（RNN）ベースの言語モデルよりも長期的な依存関係をモデル化します。また、マスク言語モデリング（MLM）タスクも導入されました。このタスクでは、すべてのトークンのランダムな15%がマスクされ、モデルによって予測され、真の双方向性が実現されます。

金融感情の分析は、専門的な言語と、その分野のラベル付けされたデータが不足しているために困難になっています。 "FinBert"事前に訓練されたBERTに基づく言語モデルであるBERTは、ドメインに適合しています "ロイターTRRC2"、金融コーパス、およびラベル付けされたデータと微調整された("金融PhraseBankの"）を参照してください。研究者たちは、財務用語を使ってニュース記事から4、500件の文章を抽出した。次に、16人の専門家と修士の学生が、財務上の背景にポジティブ、ニュートラル、ネガティブのラベルを付けています。2016年から2020年までの間に、FinBERTと他の2つの事前トレーニングパイプライン（）を使用してNASDAQ企業収益の売上高記録の感情を分析するために、Sparkのエンドツーエンドのワークフローを構築しました "財務ニュースの業況分析"、 "ドキュメントDLについて説明する"）をSparkのNLPから取得します。

Spark NLPの基礎となるディープラーニングエンジンは、機械学習向けのエンドツーエンドのオープンソースプラットフォームであるTensorFlowです。モデル構築が容易で、どこでも堅牢なML生産を実現し、研究のための強力な実験を可能にします。このため、Sparkの「yarn cluster」モードでパイプラインを実行する場合、基本的には分散TensorFlowを実行し、1つのマスターノードと複数のワーカーノード、およびクラスタにマウントされたネットワーク接続型ストレージにわたって、データとモデルの並列化を行いました。

Horovodの分散トレーニング

MapReduce関連のパフォーマンスの中核となるHadoop検証は、TeraGen、TeraSort、TeraValidate、およびDFSIO（読み取りおよび書き込み）を使用して実行されます。に、TeraGenおよびTeraSortの検証結果を示します "TR-3969：『NetApp Solutions for Hadoop』" Eシリーズおよび「ストレージ階層化」（xref）for AFFのセクションに記載されています。

お客様からの要望に基づいて、Sparkを使用したトレーニングの配布は、さまざまなユースケースで最も重要なトレーニングの1つと考えています。このドキュメントでは、を使用しました "Hovorod on Spark（SparkでのHovorod" ネットアップのオンプレミス、クラウドネイティブ、ハイブリッドクラウドソリューションでSparkのパフォーマンスを検証するには、AFF ストレージコントローラ、Azure NetApp Files 、FAS StorageGRID をご利用ください。

Horovod on Sparkパッケージは、Horovodの便利なラッパーを提供します。このラッパーはSparkクラスタで

分散されたトレーニングワークロードを簡単に実行できるようにするものです。厳密なモデル設計ループでは、トレーニングデータと推論データが存在するSparkで、データ処理、モデルトレーニング、モデル評価がすべて行われます。

SparkでHorovodを実行するためのAPIには、高レベルのエスティメータAPIと低レベルの実行APIの2つがあります。どちらも、Sparkの実行者に対してHorovodを起動するために同じ基盤メカニズムを使用していますが、Estimator APIはデータ処理、モデルトレーニングループ、モデルチェックポイント、メトリック収集、および分散トレーニングを抽象化します。Horovod Spark Estimators、TensorFlow、Kerasを使用して、に基づくエンドツーエンドのデータ準備と分散トレーニングワークフローを実施しました "Kagle Rossmann Store Sales" 競合他社

スクリプト「kers_spark_horovod_Rossmann _ estimator.py」は、のセクションにあります "主なユースケースごとにPythonスクリプトを使用できます。" 次の3つの部分で構成されます

- 最初の部分では、Kaggleが提供し、コミュニティが収集した最初のCSVファイルのセットを介して、さまざまなデータ前処理ステップを実行します。入力データは'Validation'サブセットとテストデータセットで構成されるトレーニングセットに分かれています
- 2番目の部分では、対数シグスモイド活性化関数とAdamオプティマイザを使用してKeras Deep Neural Network（DNN）モデルを定義し、Sparkに対してHorovodを使用してモデルの分散トレーニングを実行します。
- 3番目の部分では、検証セット全体の平均絶対エラーを最小化する最適なモデルを使用して、テストデータセットの予測を実行します。次に、出力CSVファイルが作成されます。

を参照してください "「機械学習」" を参照してください。

CTR予測にKerasを使用したマルチワーカーディープラーニング

ML プラットフォームとアプリケーションの最近の進歩により、現在は大規模な学習が注目されています。クリックスルー率（ CTR ）は、オンライン広告インプレッション数 100 件あたりの平均クリックスルー数（パーセンテージ）と定義されています。デジタルマーケティング、小売、 E コマース、サービスプロバイダなど、さまざまな業界やユースケースで重要な指標として広く採用されています。を参照してください "TR-4904 ：『 Distributed Training in Azure - Click Through Rate Prediction 』" Kubernetes、分散データETL、DaskおよびCUDA MLを使用したモデルトレーニングなど、CTRのアプリケーションとエンドツーエンドのクラウドAIワークフロー実装の詳細を確認できます。

このテクニカルレポートでは、別のを使用しました "Criteo Terabyteのログデータセットをクリックします"（TR-4904を参照）。Kerasを使用した複数のワーカーによる分散型ディープラーニングで、Deep NetworkモデルとCross Network（DCN）モデルを使用したSparkワークフローを構築し、ログ損失エラー機能のパフォーマンスをベースラインSparkのSpark ML Logistic Regression(ログ記録的回帰)モデルと比較します。DCNは、制限された角度の効率的な機能の相互作用を効率的にキャプチャし、高度な非線形相互作用を学習し、手動によるフィーチャーエンジニアリングや完全な検索を必要とせず、計算コストも低くなります。

Webスケールのレコメンダシステムのデータはほとんどが個別に分類されるため、フィーチャーの探索には困難な大規模でスパースな機能スペースが必要になります。これは、大規模なシステムのほとんどをロジスティック回帰などの線形モデルに限定しています。しかし、予測可能な機能を頻繁に特定すると同時に、見過ごしていない機能やまれなクロス機能を調べることが、予測を適切に行うための鍵となります。線形モデルは単純で、解析可能で、簡単にスケール変更できますが、表現力は限られています。

一方、クロス機能は、モデルの表現力を向上させる上で重要であることが示されています。残念なことに、このような機能を特定するには、手動での機能開発や完全な検索が必要になることがよくあります目に見えない機能の相互作用を一般化することは、しばしば困難です。DCNのような十字型ニューラルネットワークを使用すると、自動で機能交差を明示的に適用することで、タスク固有の機能エンジニアリングを回避できます。

クロスネットワークは複数のレイヤで構成されており、レイヤの深さによって高度な相互作用がプロバンスされます。各レイヤは、既存のレイヤに基づいて上位の相互作用を生成し、以前のレイヤからの相互作用を保持します。

Deep Neural Network（DNN；ディープニューラルネットワーク）は、さまざまな機能で非常に複雑なインタラクションをキャプチャすることを約束します。ただし、DCNと比較して、必要なパラメータの数は非常に多く、クロス機能を明示的に形成できず、一部のタイプの機能の相互作用を効率的に学習できない場合があります。クロスネットワークはメモリ効率が高く、実装も簡単です。クロスコンポーネントとDNNコンポーネントを共同でトレーニングし、予測機能のインタラクションを効率的に取り込み、Crito CTRデータセットで最先端のパフォーマンスを提供します。

DCNモデルは、埋め込みレイヤーとスタッキングレイヤーから始まり、クロスネットワークとディープネットワークが並行して使用されます。次に、2つのネットワークからの出力を組み合わせた最終的な組み合わせレイヤを示します。入力データは、スパースフィーチャーとデンスフィーチャーを持つベクトルにすることができます。Sparkでは、その両方です "ml" および "ミリリブ" ライブラリには「SparseVector」タイプが含まれます。したがって、ユーザーがそれぞれの機能やメソッドを呼び出す際には、2つの機能を区別し、注意することが重要です。CTR予測などのWebスケールの推薦システムでは、入力は主に「country = USA」などの分類的な機能です。このような機能は、多くの場合、1つのホットベクトルとしてエンコードされます。たとえば、「[0,1,0,..]」のようになります。「SparseVector」を使用したワン・ホット・エンコーディング（OHE）は、絶えず変化する語彙や拡大する語彙を持つ実世界のデータセットを扱う場合に便利です。で例を変更しました "Deepctr" 大きなボキャブラリを処理するために、DCNの埋め込みレイヤーとスタッキングレイヤーに埋め込みベクトルを作成します。

。 "Critoディスプレイ広告のデータセット" 広告のクリックスルーレートを予測します。13の整数型の機能と、各カテゴリの基数が多い26の分類的な機能があります。このデータセットでは、入力サイズが大きいため、ログロスの0.001が実質的に大きく改善されています。大規模なユーザベースの予測精度がわずかに向上すると、企業の収益が大きく増加する可能性があります。データセットには7日間の11GBのユーザログが格納されており、これは約4100万レコードに相当します。Sparkのdataframe .randomSplit()関数を使用して、トレーニング用のデータ(80%)、クロス検証(10%)、およびテスト用の残りの10%をランダムに分割しました。

DCNは、Kerasを使用したTensorFlowに実装されました。DCNを使用したモデルトレーニングプロセスの実装には、次の4つの主要コンポーネントがあります。

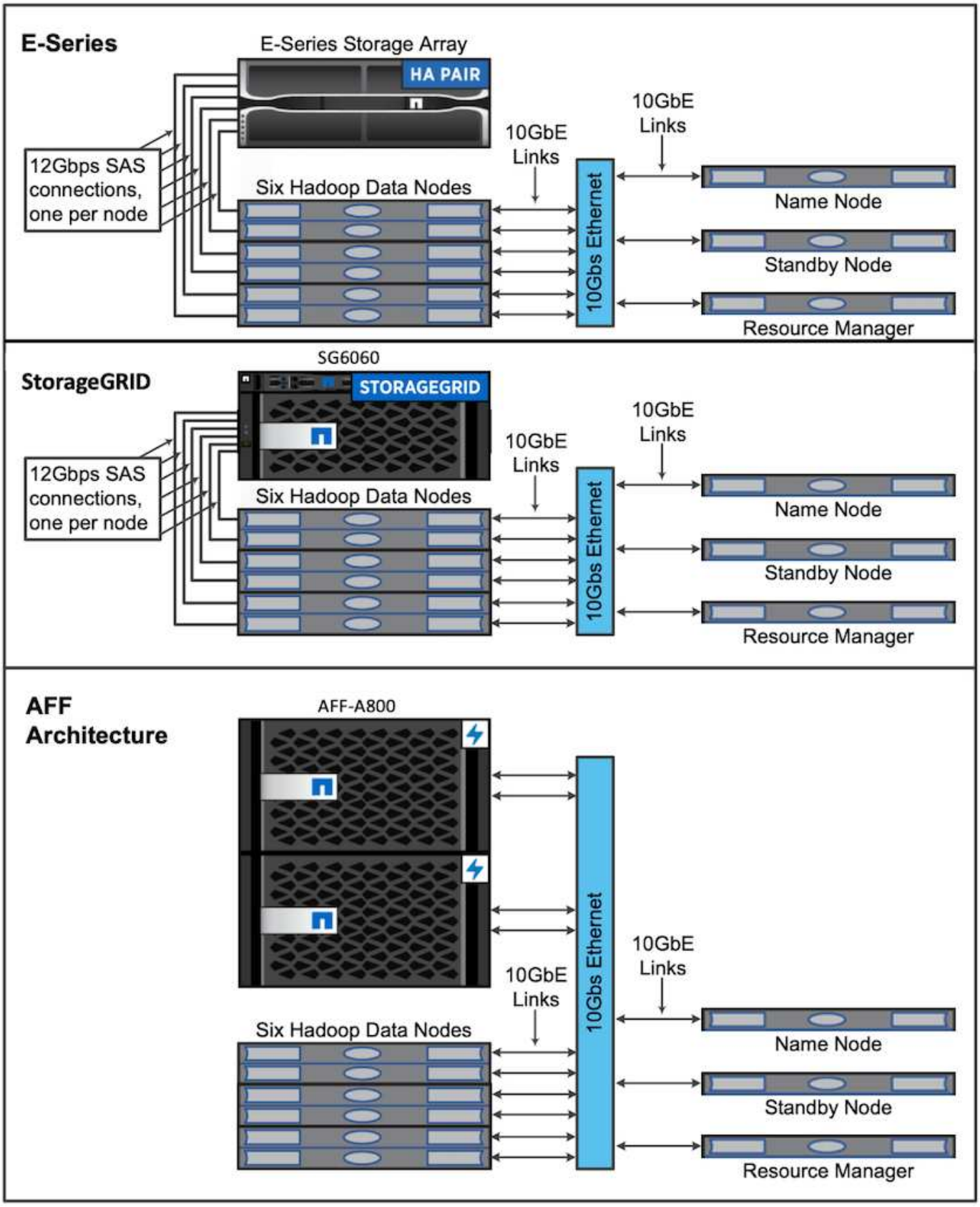
- *データ処理と埋め込み。*ログトランスフォームを適用することで、リアルタイム機能が正規化されます。カテゴリフィーチャーの場合、寸法6×（カテゴリの基数）1/4の密度の高いベクトルにフィーチャーを埋め込みます。すべての埋め込み結果を次元1026のベクトルに連結します。
- 最適化 Adam Optimizerを使用してミニバッチ確率的最適化を適用しました。バッチサイズは512に設定されています。ディープネットワークにバッチ正規化が適用され、グラジエントクリップの基準が100に設定されました。
- *均一化。*私達はL2の均一化かドロップアウトが有効であることが見つけられなかったので早い停止を使用した。
- * Hyperparameters*。非表示レイヤー数、非表示レイヤーサイズ、初期学習レート、およびクロスレイヤー数に基づくグリッド検索に基づく結果を報告します。非表示レイヤーの数は2〜5で、非表示レイヤーのサイズは32〜1024です。DCNの場合、クロスレイヤの数は1〜6です。初期学習レートは0.0001から0.001に調整され、0.0001単位で増加しました。すべての実験は訓練ステップ150,000で早期停止を適用し、それを超えて過剰なフィッティングが発生し始めました。

DCNに加えて、CTRの予測に使用される他の一般的なディープラーニングモデルもテストしました "DeepFM"、 "xDeepFM"、 "自動内部（AutoInt"および "DCN v2"。

検証に使用するアーキテクチャ

この検証では、4つのワーカーノードと1つのマスターノードにAFF-A800 HAペアを使用しました。すべてのクラスタ・メンバーを、10GbEネットワーク・スイッチを介して接続しました。

今回のNetApp Sparkの解決策 検証では、E5760、E5724、AFF-A800の3種類のストレージコントローラを使用しました。Eシリーズストレージコントローラは、12Gbps SAS接続の5つのデータノードに接続されました。AFF のHAペアストレージコントローラは、エクスポートされたNFSボリュームを10GbEでHadoopワーカーノードに接続することで提供します。Hadoopクラスタのメンバーは、Eシリーズ、AFF 、およびStorageGRID のHadoopソリューションで10GbE接続を介して接続されます。



テスト結果

TeraGenベンチマークツールでTeraSortおよびTeraValidateスクリプトを使用し

て、E5760、E5724、およびAFF-A800構成でのSparkのパフォーマンス検証を測定しました。さらに、SparkのNLPパイプラインとTensorFlow分散トレーニング、Horovod分散トレーニング、Kerasを使用したディープFMでのCTR予測を利用した複数ワーカーのディープラーニングという、3つの主なユースケースをテストしました。

EシリーズとStorageGRID の検証には、Hadoopレプリケーションファクタ2を使用しました。AFF の検証に使用したデータソースは1つだけです。

次の表に、Sparkのパフォーマンス検証のハードウェア構成を示します。

を入力します	**Hadoop**ワーカーノード	ドライブタイプ	ノードあたりのドライブ数	ストレージコントローラ
SG6060 の設計	4.	（ SAS ）。	12.	単一のハイアベイラビリティ（HA）ペア
E5760	4.	（ SAS ）。	60	単一のHAペア
E5724	4.	（ SAS ）。	24	単一のHAペア
AFF800が必要です	4.	SSD の場合	6.	単一のHAペア

次の表に、ソフトウェア要件を示します。

ソフトウェア	バージョン
RHEL	7.9
OpenJDKランタイム環境	1.8.0
OpenJDK 64ビットのServer VM	25.302
Git	2.24.1
GCC/G++	11.2.1.
火花	3.2.1
PySpark	3.1.2
SparkNLPです	3.4.2
TensorFlowです	2.9.0
クラス	2.9.0
ホロボド	0.24.3

財務心理分析

公開しました "TR-4910 ：『 NetApp AI と顧客コミュニケーションを組み合わせた感情分析』"を使用して、エンドツーエンドの会話型AIパイプラインを構築しました "NetApp DataOps ツールキット"、AFF ストレージ、NVIDIA DGXシステムパイプラインは、DataOpsツールキットを活用して、バッチオーディオ信号処理、Automatic Speech Recognition（ASR）、転送学習、感情分析を実行します。 "NVIDIA Riva SDKを参照してください"および "Taoフレームワーク"。金融サービス業界へのセンチメント分析のユースケースの拡大、SparkNLPワークフローの構築、特定事業体の認識など、さまざまなNLPタスクに対する3つのBERTモデルのロード、NASDAQ Top 10 Companiesの四半期収益コールに関するセンテンスレベルのセンチメントの取

得。

次のスクリプト「センチメント_アナリシス_スパーク」。PY'はFinBERTモデルを使用してHDFS内のトランスクリプトを処理し、次の表に示すようにポジティブでニュートラルでネガティブな感情カウントを生成します。

```
-bash-4.2$ time ~/anaconda3/bin/spark-submit
--packages com.johnsnowlabs.nlp:spark-nlp_2.12:3.4.3
--master yarn
--executor-memory 5g
--executor-cores 1
--num-executors 160
--conf spark.driver.extraJavaOptions="-Xss10m -XX:MaxPermSize=1024M"
--conf spark.executor.extraJavaOptions="-Xss10m -XX:MaxPermSize=512M"
/sparkusecase/tr-4570-nlp/sentiment_analysis_spark.py
hdfs:///data1/Transcripts/
> ./sentiment_analysis_hdfs.log 2>&1
real13m14.300s
user557m11.319s
sys4m47.676s
```

次の表に、2016年から2020年までのNASDAQトップ10企業の収益/コール、センテンスレベルの感情分析を示します。

センチメントの数値と割合	**10**社すべて	**AAPL**	**AMD**	**AMZN**	**CSCO**	**GOOGL**	**INTC**	マイクロソフト	**NVDA**
正の数	7447	1567年	743	290	682	826	824	904	417
ニュートラルカウント	64067	6856	7596	5086	6650	5914	6099	5715	6189
負の数	1787年になります	253	213	84	189	97	282	202.	89
分類なしのカウント	196	0	0	76	0	0	0	1.	0
（合計数）	73497	8676	8552.	5536	7521	6837	7205.	6822	6695

割合の点では、CEOとCFOが話している文のほとんどは事実上であり、中立的な感情を持っています。決算発表時に、アナリストは肯定的または否定的な感情を伝える可能性のある質問をします。また、マイナスやプラスの心理が株価に与える影響についても、取引日の同日または翌日に定量的に調査することもできます。

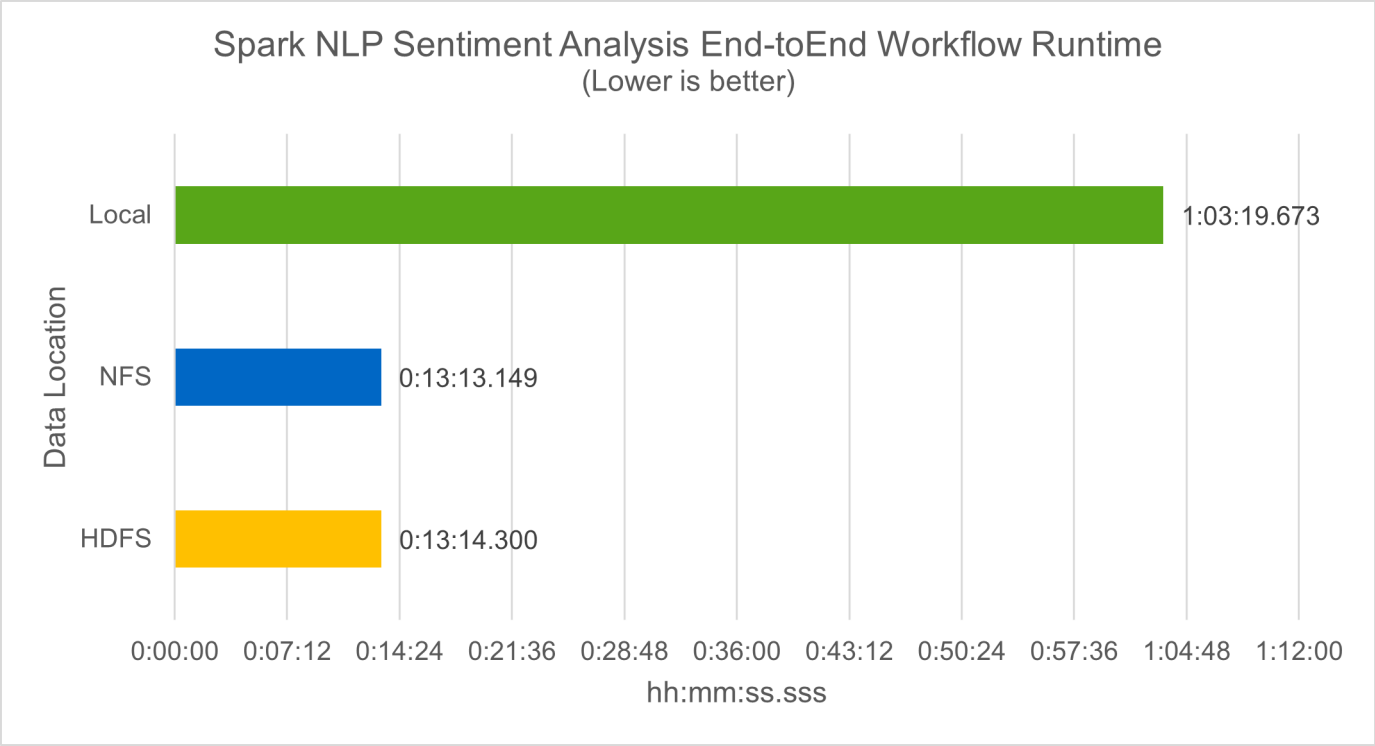
次の表に、NASDAQトップ10企業の文章レベルの感情分析をパーセントで示します。

センチメントの割合	**10**社すべて	**AAPL**	**AMD**	**AMZN**	**CSCO**	**GOOGL**	**INTC**	マイクロソフト	**NVDA**
肯定的	10.13%	18.06%	8.69%	5.24%	9.07%	12.08%	11.44%	13.25%	6.23%
ニュートラル	87.17%	79.02%	88.82%	91.87%	88.42%	86.50%	84.65%	83.77%	92.44%
負	2.43%	2.92%	2.49%	1.52%	2.51 %	1.42 %	3.91%	2.96 %	1.33%
分類なし	0.27%	0%	0%	1.37 %	0%	0%	0%	0.01%	0%

ワークフローの実行時間に関しては'local'モードからHDFSの分散環境に至るまで4.78倍の大幅な改善が見られましたまた'NFSを活用することで'さらに0.14%の向上が見られました

```
-bash-4.2$ time ~/anaconda3/bin/spark-submit
--packages com.johnsnowlabs.nlp:spark-nlp_2.12:3.4.3
--master yarn
--executor-memory 5g
--executor-cores 1
--num-executors 160
--conf spark.driver.extraJavaOptions="-Xss10m -XX:MaxPermSize=1024M"
--conf spark.executor.extraJavaOptions="-Xss10m -XX:MaxPermSize=512M"
/sparkusecase/tr-4570-nlp/sentiment_analysis_spark.py
file:///sparkdemo/sparknlp/Transcripts/
> ./sentiment_analysis_nfs.log 2>&1
real13m13.149s
user537m50.148s
sys4m46.173s
```

次の図に示すように、データとモデルの並列処理によって、データ処理と分散TensorFlowモデルの推論速度が向上しています。NFSのデータの場所では、トレーニング済みのモデルがワークフローのボトルネックになっているため、ランタイムが若干向上しました。Transcriptデータセットのサイズを増やすと、NFSの方が明らかになります。



Horovodのパフォーマンスを使用した分散トレーニング

次のコマンドでは、1つのコアを持つ160個の実行者を持つ単一の「マスター」ノードを使用して、Sparkクラスタ内にランタイム情報とログファイルを生成しました。実行者メモリはメモリ不足エラーを回避するために5GBに制限されていました。を参照してください "「主要な各ユースケース用のPythonスクリプト」" データ処理、モデル・トレーニング、およびモデル精度計算の詳細については、「kers_spark_horovod_Rossmann _dimator.py」を参照してください。

```
(base) [root@n138 horovod]# time spark-submit
--master local
--executor-memory 5g
--executor-cores 1
--num-executors 160
/sparkusecase/horovod/keras_spark_horovod_rossmann_estimator.py
--epochs 10
--data-dir file:///sparkusecase/horovod
--local-submission-csv /tmp/submission_0.csv
--local-checkpoint-file /tmp/checkpoint/
> /tmp/keras_spark_horovod_rossmann_estimator_local. log 2>&1
```

トレーニング期間が10回の場合の結果、次のようになりました。

```
real43m34.608s
user12m22.057s
sys2m30.127s
```

入力データの処理、DNNモデルのトレーニング、精度の計算、TensorFlowチェックポイントと予測結果のCSVファイルの作成に43分以上かかりました。トレーニング期間を10に制限しました。実際には100に設定されていることが多く、モデルの精度が十分であることを確認しています。トレーニング時間は通常、エポックの数に比例して拡大します。

次に、クラスタで使用可能な4つのワーカーノードを使用して、HDFS内のデータで「yarn」モードで同じスクリプトを実行しました。

```
(base) [root@n138 horovod]# time spark-submit
--master yarn
--executor-memory 5g
--executor-cores 1 --num-executors 160
/sparkusecase/horovod/keras_spark_horovod_rossmann_estimator.py
--epochs 10
--data-dir hdfs:///user/hdfs/tr-4570/experiments/horovod
--local-submission-csv /tmp/submission_1.csv
--local-checkpoint-file /tmp/checkpoint/
> /tmp/keras_spark_horovod_rossmann_estimator_yarn.log 2>&1
```

結果として得られる実行時間は次のように改善されました。

```
real8m13.728s
user7m48.421s
sys1m26.063s
```

HorovodのモデルとSparkのデータの並列化により、「yarn」と「local」モードを比較したランタイムが5.29x短縮され、トレーニング期間が10時間に短縮されました。次の図に、凡例に「hdfs」と「Local」を示します。基盤となるTensorFlow DNNモデルのトレーニングを、GPUがあればさらに高速化できます。このテストを実施し、今後のテクニカルレポートに結果を公開する予定です。

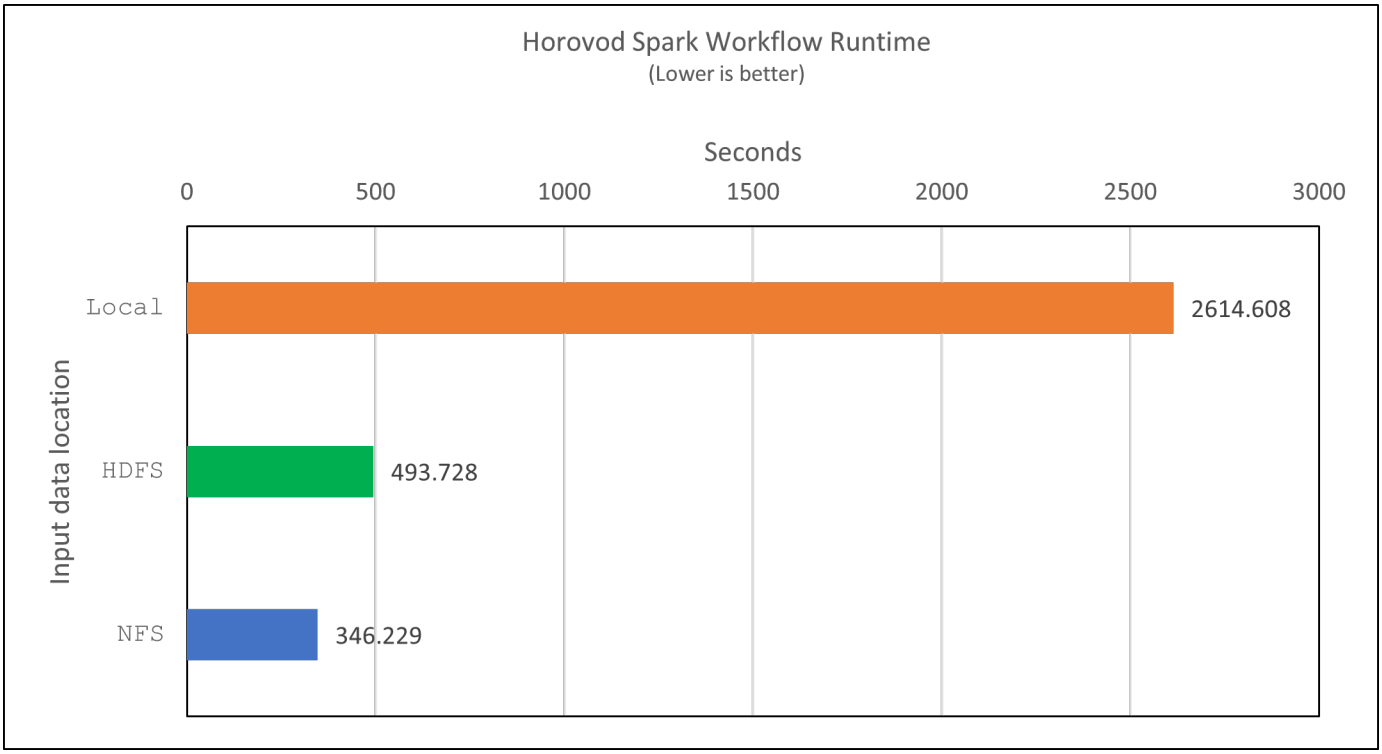
次のテストでは、NFSとHDFSの入力データをランタイムで比較しました。AFF A800のNFSボリュームは、Sparkクラスタ内の5つのノード（マスター1つ、ワーカー4つ）にまたがって「/sparkdemo/horovod」にマウントされました。前のテストと同様のコマンドを実行しましたが'--data-dir'パラメータは現在NFSマウントを指しています

```
(base) [root@n138 horovod]# time spark-submit
--master yarn
--executor-memory 5g
--executor-cores 1
--num-executors 160
/sparkusecase/horovod/keras_spark_horovod_rossmann_estimator.py
--epochs 10
--data-dir file:///sparkdemo/horovod
--local-submission-csv /tmp/submission_2.csv
--local-checkpoint-file /tmp/checkpoint/
> /tmp/keras_spark_horovod_rossmann_estimator_nfs.log 2>&1
```

NFSを使用した場合の実行時間は次のようになりました。

```
real 5m46.229s
user 5m35.693s
sys  1m5.615s
```

次の図に示すように、1.43倍の速度がさらに向上しました。このため、ネットアップのオールフラッシュストレージをクラスタに接続することで、Horovod Sparkワークフローの高速データ転送と配信というメリットを享受し、1つのノードで実行する場合と比べて7.55x高速化を達成できます。



CTR予測パフォーマンスのディープラーニングモデル

クリック率を最大化するように設計されたレコメンダシステムでは、低い順に数学的に計算される可能性のあ

る、ユーザーの行動の背後にある高度な機能の相互作用を学習する必要があります。低次と高次の両方の機能の相互作用は、どちらか一方をバイアスすることなく、ディープラーニングモデルにとっても同様に重要です。新しいニューラルネットワークアーキテクチャでの機能学習に向けて、界面活性化機械ベースのニューラルネットワークであるDeep Factorization Machine（DeepFM）は、界面活性化装置を組み合わせた推奨製品です。

従来の三角分解機械は、機能間の潜伏ベクトルの内側としてのペアワイズ機能の相互作用をモデル化しますが、理論的には高次情報をキャプチャすることができます。実際には、機械学習の実践では、一般的に、計算と保管の複雑さが高いため、二次フィーチャーの相互作用しか使用しませGoogleなどのディープニューラルネットワークのバリエーション "ワイド・モデルとディープ・モデル" 一方、リニアワイドモデルとディープモデルを組み合わせて、ハイブリッドネットワーク構造で高度な機能の相互作用を学習します。

このワイド・ディープ・モデルには2つの入力があります。1つは基本的なワイド・モデル用で、もう1つはディープのためです。後者の部分では、エキスパートフィーチャー・エンジニアリングが必要です。このため、この手法は他のドメインには一般的にできません。ワイド・ディープ・モデルとは異なり、DeepFMはフィーチャー・エンジニアリングなしでRAW機能を使用して効率的にトレーニングできます。ワイド・パートとディープ・パートは同じ入力と埋め込みベクトルを共有するためです。

私たちはまず'セクションのrun_classification_Crito_spark.pyを使用して'ctr_trine.csv'という名前のCSVファイルにCrito'trine.csv'をNFSマウント'/sparkdemo/tr-4570-data'に格納しました "「主なユースケースごとにPythonスクリプトを用意しています。」" このスクリプト内で'関数process_input_file'は'タブを削除し'区切り文字として"を'改行として"を挿入するための複数の文字列メソッドを実行しますコードブロックがコメントとして表示されるように、元の「train .txt」を1回だけ処理する必要があることに注意してください。

以下の異なるDLモデルのテストでは、「ctr_train .csv」を入力ファイルとして使用しました。その後のテスト実行では、入力CSVファイルがSpark DataFrameに読み込まれ、スキーマに「label」のフィールド、整数の高密度フィーチャー「I1」、「I2」、「I3」、…、「I13」が含まれています。 また'希薄な機能は"C1'、'C2'、'C3'、…、'C26']`です次の「spark-smSubmit」コマンドは、入力CSVで実行し、クロス検証用に20%のスプリットを備えたDeepFMモデルをトレーニングし、10回のトレーニング期間後に最適なモデルを選択して、テストセットの予測精度を計算します。

```
(base) [root@n138 ~]# time spark-submit --master yarn --executor-memory 5g
--executor-cores 1 --num-executors 160
/sparkusecase/DeepCTR/examples/run_classification_criteo_spark.py --data
-dir file:///sparkdemo/tr-4570-data >
/tmp/run_classification_criteo_spark_local.log 2>&1
```

データファイル「ctr_train .csv」は11 GBを超えるため、エラーを回避するには、データセットサイズよりも十分な「spark.driver.maxResultSize」を設定する必要があります。

```
 spark = SparkSession.builder \
    .master("yarn") \
    .appName("deep_ctr_classification") \
    .config("spark.jars.packages", "io.github.ravwojdyla:spark-schema-
utils_2.12:0.1.0") \
    .config("spark.executor.cores", "1") \
    .config('spark.executor.memory', '5gb') \
    .config('spark.executor.memoryOverhead', '1500') \
    .config('spark.driver.memoryOverhead', '1500') \
    .config("spark.sql.shuffle.partitions", "480") \
    .config("spark.sql.execution.arrow.enabled", "true") \
    .config("spark.driver.maxResultSize", "50gb") \
    .getOrCreate()
```

上記のSparkSession.Builder'構成でも有効になっています "Apache Arrowの"は、SparkのDataFrameを「df.toPandas ()」メソッドを使用してPandas DataFrameに変換します。

```
22/06/17 15:56:21 INFO scheduler.DAGScheduler: Job 2 finished: toPandas at
/sparkusecase/DeepCTR/examples/run_classification_criteo_spark.py:96, took
627.126487 s
Obtained Spark DF and transformed to Pandas DF using Arrow.
```

ランダムにスプリットした後、トレーニングデータセットに3、6M行以上、テストセットに9、000サンプル以上が存在します。

```
Training dataset size =  36672493
Testing dataset size =  9168124
```

このテクニカルレポートでは、GPUを使用せずにCPUテストに焦点を当てているため、適切なコンパイラフラグを使用してTensorFlowを構築することが重要です。これにより、GPUアクセラレーションライブラリの呼び出しを回避し、TensorFlowのAdvanced Vector Extensions（AVX）およびAVX2命令を最大限に活用できます。これらの機能は、ベクトル化された加算、フィードフォワード内の行列乗算、またはバック伝播DNNトレーニングなどの線形代数計算用に設計されています。256ビット浮動小数点(FP)レジスタを使用したAVX2で使用可能なFMA (fMultiply Add)命令は、整数コードとデータ型に最適で、最大2倍の速度を実現します。FPコードとデータ型の場合、AVX2はAVXと比較して8%の高速化を実現します。

```
2022-06-18 07:19:20.101478: I
tensorflow/core/platform/cpu_feature_guard.cc:151] This TensorFlow binary
is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the
following CPU instructions in performance-critical operations:  AVX2 FMA
To enable them in other operations, rebuild TensorFlow with the
appropriate compiler flags.
```

ソースからTensorFlowを構築する場合は、を使用することを推奨します "バザー"。今回の環境では、シェルプロンプトで以下のコマンドを実行して、「リタイア」、「リタイヤ」、「バザール」をインストールしました。

```
yum install dnf
dnf install 'dnf-command(copr)'
dnf copr enable vbatts/bazel
dnf install bazel5
```

ビルドプロセス中にC++ 17の機能を使用するには、GCC 5以降を有効にする必要があります。この機能は、RHELがソフトウェアコレクションライブラリ(SCL)とともに提供します。次のコマンドは'devtoolset'とGCC 11.2.1をRHEL 7.9クラスタにインストールします

```
subscription-manager repos --enable rhel-server-rhscl-7-rpms
yum install devtoolset-11-toolchain
yum install devtoolset-11-gcc-c++
yum update
scl enable devtoolset-11 bash
. /opt/rh/devtoolset-11/enable
```

最後の2つのコマンドは'devtoolsets-11'を有効にしますこれには'/opt/r/devtoolset11-root//usr/bin/gcc`(GCC 11.2.1)が使用されますまた'gitのバージョンが1.8.3よりも大きいことを確認してください(RHEL 7.9に付属しています)これを参照してください "記事" 「git」を2.24.1に更新します。

最新のTensorFlowマスターリポジトリもすでにクローニング済みであるとします。次に'workspace'ファイルを使用して'workspace'ディレクトリを作成し'AVX、AVX2、FMAを使用してソースからTensorFlowを構築しますconfigureファイルを実行し、正しいPythonバイナリの場所を指定します。 "CUDA （ CUDA" はGPUを使用していないため、テストでは無効になっています。設定に応じて'`. bazelrc'ファイルが生成されますさらに、ファイルを編集し、HDFSのサポートを有効にするために「build—define=no_hdfs_support=false」を設定しました。セクションの「. bazelrc」を参照してください "「主なユースケースごとにPythonスクリプトを用意しています。」" 設定とフラグの完全なリストについては、を参照してください。

```
./configure
bazel build -c opt --copt=-mavx --copt=-mavx2 --copt=-mfma --copt=
-mfpmath=both -k //tensorflow/tools/pip_package:build_pip_package
```

適切なフラグを使用してTensorFlowを構築したら、次のスクリプトを実行してCritoディスプレイ広告データセットを処理し、DeepFMモデルをトレーニングし、予測スコアからReceiver Operating Characteristic Curve（ROC AUC）の下の領域を計算します。

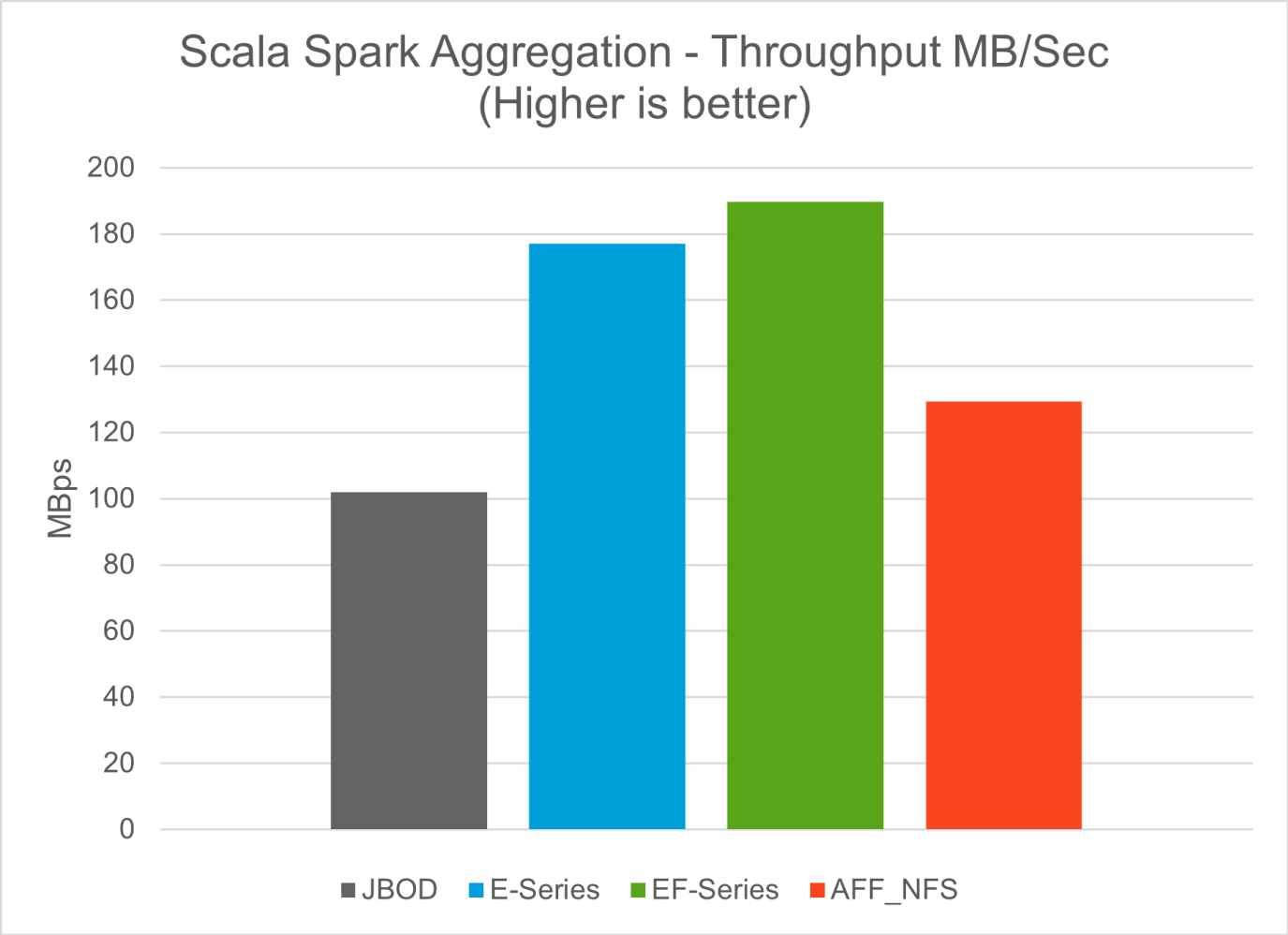
```
(base) [root@n138 examples]# ~/anaconda3/bin/spark-submit
--master yarn
--executor-memory 15g
--executor-cores 1
--num-executors 160
/sparkusecase/DeepCTR/examples/run_classification_criteo_spark.py
--data-dir file:///sparkdemo/tr-4570-data
> . /run_classification_criteo_spark_nfs.log 2>&1
```

トレーニング期間が10回終了したら、テストデータセットのAUCスコアを取得しました。

```
Epoch 1/10
125/125 - 7s - loss: 0.4976 - binary_crossentropy: 0.4974 - val_loss:
0.4629 - val_binary_crossentropy: 0.4624
Epoch 2/10
125/125 - 1s - loss: 0.3281 - binary_crossentropy: 0.3271 - val_loss:
0.5146 - val_binary_crossentropy: 0.5130
Epoch 3/10
125/125 - 1s - loss: 0.1948 - binary_crossentropy: 0.1928 - val_loss:
0.6166 - val_binary_crossentropy: 0.6144
Epoch 4/10
125/125 - 1s - loss: 0.1408 - binary_crossentropy: 0.1383 - val_loss:
0.7261 - val_binary_crossentropy: 0.7235
Epoch 5/10
125/125 - 1s - loss: 0.1129 - binary_crossentropy: 0.1102 - val_loss:
0.7961 - val_binary_crossentropy: 0.7934
Epoch 6/10
125/125 - 1s - loss: 0.0949 - binary_crossentropy: 0.0921 - val_loss:
0.9502 - val_binary_crossentropy: 0.9474
Epoch 7/10
125/125 - 1s - loss: 0.0778 - binary_crossentropy: 0.0750 - val_loss:
1.1329 - val_binary_crossentropy: 1.1301
Epoch 8/10
125/125 - 1s - loss: 0.0651 - binary_crossentropy: 0.0622 - val_loss:
1.3794 - val_binary_crossentropy: 1.3766
Epoch 9/10
125/125 - 1s - loss: 0.0555 - binary_crossentropy: 0.0527 - val_loss:
1.6115 - val_binary_crossentropy: 1.6087
Epoch 10/10
125/125 - 1s - loss: 0.0470 - binary_crossentropy: 0.0442 - val_loss:
1.6768 - val_binary_crossentropy: 1.6740
test AUC 0.6337
```

以前のユースケースと同様に、Sparkワークフローランタイムを異なる場所にあるデータと比較しました。次

の図は、Sparkワークフローランタイムのディープラーニングの CTR予測の比較を示しています。

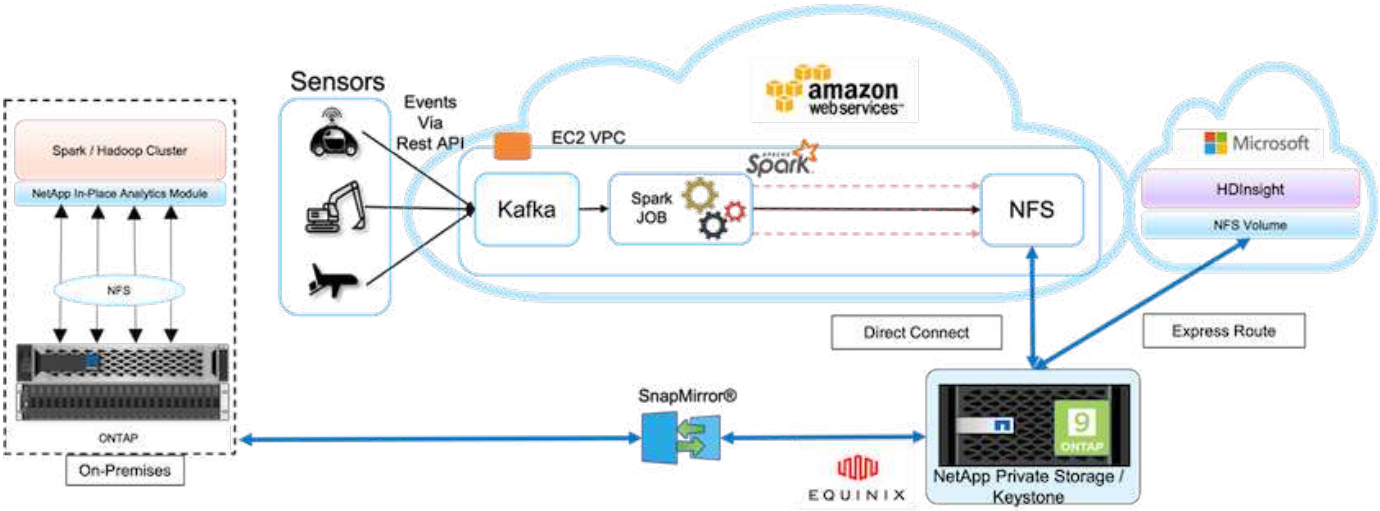


Hybrid Cloud解決策 の略

最新のエンタープライズデータセンターとは、複数の分散インフラ環境を、オンプレミスや複数のパブリッククラウドと一貫した運用モデルで継続的なデータ管理プレーンを通じて接続するハイブリッドクラウドです。ハイブリッドクラウドを最大限に活用するには、データの変換やアプリケーションのリファクタリングを行わなくても、オンプレミス環境とマルチクラウド環境間でデータをシームレスに移動できる必要があります。

データ保護などのユースケースでセカンダリストレージをクラウドに移行するか、アプリケーション開発やDevOpsなどのビジネスクリティカルなワークロードをクラウドに移行することで、ハイブリッドクラウドへの移行を開始するとお客様から指摘を受けています。そして、より重要なワークロードに移行します。Webおよびコンテンツホスティング、DevOpsやアプリケーション開発、データベース、分析、コンテナ化されたアプリケーションは、最も一般的なハイブリッドクラウドワークロードです。エンタープライズAIプロジェクトの複雑さ、コスト、リスクは、これまでAIの導入を試験段階から本番運用に妨げていました。

ネットアップのハイブリッドクラウド解決策 なら、セキュリティ、データガバナンス、コンプライアンスの統合ツールを活用できます。また、分散環境全体でデータとワークフローを一元管理できる単一のコントロールパネルを使用して、総所有コストを消費量に応じて最適化できます。次の図は、お客様のビッグデータ分析データにマルチクラウド接続を提供するという課題を抱えるクラウドサービスパートナーの解決策 の例です。



このシナリオでは、AWSでさまざまなソースから受け取ったIoTデータが、NetApp Private Storage（NPS）内の1箇所に保存されます。NPSストレージは、AWSとAzure上にあるSparkクラスタやHadoopクラスタに接続されているため、複数のクラウドで同じデータにアクセスするビッグデータ分析アプリケーションを実行できます。このユースケースの主な要件と課題は次のとおりです。

- お客様は、複数のクラウドを使用して、同じデータに対して分析ジョブを実行したいと考えています。
- オンプレミス環境やクラウド環境などのさまざまなソースから、さまざまなセンサーやハブを介してデータを受信する必要があります。
- 解決策 は、効率性とコスト効率に優れている必要があります。
- 主な課題は、コスト効率と効率に優れた解決策 を構築し、オンプレミス環境とクラウド環境の間でハイブリッド分析サービスを提供することです。

ネットアップのデータ保護機能とマルチクラウド接続解決策 は、複数のハイパースケーラにわたるクラウド分析アプリケーションの課題を解決します。上の図に示すように、センサーからのデータはストリーミングされ、 Kafka を介して AWS Spark クラスタに取り込まれます。データは NPS 内の NFS 共有に格納されます。NPS は、 Equinix データセンター内のクラウドプロバイダの外部にあります。

NetApp NPSは、それぞれDirect Connect接続とExpress Route接続を通じてAmazon AWSとMicrosoft Azureに接続されているため、インプレース分析モジュールを利用して、AmazonとAWS両方の分析クラスタからデータにアクセスできます。そのため、オンプレミスストレージとNPSストレージの両方でONTAP ソフトウェアが実行され、 "SnapMirror" NPSデータをオンプレミスクラスタにミラーリングし、オンプレミスと複数のクラウドにわたってハイブリッドクラウド分析を実現できます。

パフォーマンスを最大限に高めるために、通常は複数のネットワークインターフェイスと直接接続またはエクスプレスルートを使用してクラウドインスタンスからデータにアクセスすることを推奨します。ネットアップには、ほかにも、などのData Moverソリューションがあります "XCP" および "BlueXPのコピーと同期" お客様がアプリケーション対応でセキュア、コスト効率に優れたハイブリッドクラウドSparkクラスタを構築できるよう支援します。

主なユースケースごとに**Python**スクリプトを使用できます

以下の3つのPythonスクリプトは、テストした3つの主なユースケースに対応しています。1つ目は’ecimation_analysis _sparknlp.pyです

```
# TR-4570 Refresh NLP testing by Rick Huang
from sys import argv
import os
import sparknlp
import pyspark.sql.functions as F
from sparknlp import Finisher
from pyspark.ml import Pipeline
from sparknlp.base import *
from sparknlp.annotator import *
from sparknlp.pretrained import PretrainedPipeline
from sparknlp import Finisher
# Start Spark Session with Spark NLP
spark = sparknlp.start()
print("Spark NLP version:")
print(sparknlp.version())
print("Apache Spark version:")
print(spark.version)
spark = sparknlp.SparkSession.builder \
     .master("yarn") \
     .appName("test_hdfs_read_write") \
     .config("spark.executor.cores", "1") \
     .config("spark.jars.packages", "com.johnsnowlabs.nlp:spark-
nlp_2.12:3.4.3")\
     .config('spark.executor.memory', '5gb') \
     .config('spark.executor.memoryOverhead','1000')\
     .config('spark.driver.memoryOverhead','1000')\
     .config("spark.sql.shuffle.partitions", "480")\
     .getOrCreate()
sc = spark.sparkContext
from pyspark.sql import SQLContext
sql = SQLContext(sc)
sqlContext = SQLContext(sc)
# Download pre-trained pipelines & sequence classifier
explain_pipeline_model = PretrainedPipeline('explain_document_dl',
lang='en').model#pipeline_sa =
PretrainedPipeline("classifierdl_bertwiki_finance_sentiment_pipeline",
lang="en")
# pipeline_finbert =
BertForSequenceClassification.loadSavedModel('/sparkusecase/bert_sequence_
classifier_finbert_en_3', spark)
sequenceClassifier = BertForSequenceClassification \
        .pretrained('bert_sequence_classifier_finbert', 'en') \
        .setInputCols(['token', 'document']) \
        .setOutputCol('class') \
        .setCaseSensitive(True) \
        .setMaxSentenceLength(512)
```

```
def process_sentence_df(data):
    # Pre-process: begin
    print("1. Begin DataFrame pre-processing...\n")
    print(f"\n\t2. Attaching DocumentAssembler Transformer to the
pipeline")
    documentAssembler = DocumentAssembler() \
        .setInputCol("text") \
        .setOutputCol("document") \
        .setCleanupMode("inplace_full")
        #.setCleanupMode("shrink", "inplace_full")
    doc_df = documentAssembler.transform(data)
    doc_df.printSchema()
    doc_df.show(truncate=50)
    # Pre-process: get rid of  blank lines
    clean_df = doc_df.withColumn("tmp", F.explode("document")) \
        .select("tmp.result").where("tmp.end !=
-1").withColumnRenamed("result", "text").dropna()
    print("[OK!] DataFrame after initial cleanup:\n")
    clean_df.printSchema()
    clean_df.show(truncate=80)
    # for FinBERT
    tokenizer = Tokenizer() \
        .setInputCols(['document']) \
        .setOutputCol('token')
    print(f"\n\t3. Attaching Tokenizer Annotator to the pipeline")
    pipeline_finbert = Pipeline(stages=[
        documentAssembler,
        tokenizer,
        sequenceClassifier
        ])
    # Use Finisher() & construct PySpark ML pipeline
    finisher = Finisher().setInputCols(["token", "lemma", "pos",
"entities"])
    print(f"\n\t4. Attaching Finisher Transformer to the pipeline")
    pipeline_ex = Pipeline() \
        .setStages([
           explain_pipeline_model,
           finisher
           ])
    print("\n\t\t\t ---- Pipeline Built Successfully ----")
    # Loading pipelines to annotate
    #result_ex_df = pipeline_ex.transform(clean_df)
    ex_model = pipeline_ex.fit(clean_df)
    annotations_finished_ex_df = ex_model.transform(clean_df)
    # result_sa_df = pipeline_sa.transform(clean_df)
    result_finbert_df = pipeline_finbert.fit(clean_df).transform(clean_df)
```

```
    print("\n\t\t\t ----Document Explain, Sentiment Analysis & FinBERT
Pipeline Fitted Successfully ----")
    # Check the result entities
    print("[OK!] Simple explain ML pipeline result:\n")
    annotations_finished_ex_df.printSchema()
    annotations_finished_ex_df.select('text',
'finished_entities').show(truncate=False)
    # Check the result sentiment from FinBERT
    print("[OK!] Sentiment Analysis FinBERT pipeline result:\n")
    result_finbert_df.printSchema()
    result_finbert_df.select('text', 'class.result').show(80, False)
    sentiment_stats(result_finbert_df)
    return
def sentiment_stats(finbert_df):
    result_df = finbert_df.select('text', 'class.result')
    sa_df = result_df.select('result')
    sa_df.groupBy('result').count().show()
    # total_lines = result_clean_df.count()
    # num_neutral = result_clean_df.where(result_clean_df.result ==
['neutral']).count()
    # num_positive = result_clean_df.where(result_clean_df.result ==
['positive']).count()
    # num_negative = result_clean_df.where(result_clean_df.result ==
['negative']).count()
    # print(f"\nRatio of neutral sentiment = {num_neutral/total_lines}")
    # print(f"Ratio of positive sentiment = {num_positive / total_lines}")
    # print(f"Ratio of negative sentiment = {num_negative /
total_lines}\n")
    return
def process_input_file(file_name):
    # Turn input file to Spark DataFrame
    print("START processing input file...")
    data_df = spark.read.text(file_name)
    data_df.show()
    # rename first column 'text' for sparknlp
    output_df = data_df.withColumnRenamed("value", "text").dropna()
    output_df.printSchema()
    return output_dfdef process_local_dir(directory):
    filelist = []
    for subdir, dirs, files in os.walk(directory):
        for filename in files:
            filepath = subdir + os.sep + filename
            print("[OK!] Will process the following files:")
            if filepath.endswith(".txt"):
                print(filepath)
                filelist.append(filepath)
```

```
    return filelist
def process_local_dir_or_file(dir_or_file):
    numfiles = 0
    if os.path.isfile(dir_or_file):
        input_df = process_input_file(dir_or_file)
        print("Obtained input_df.")
        process_sentence_df(input_df)
        print("Processed input_df")
        numfiles += 1
    else:
        filelist = process_local_dir(dir_or_file)
        for file in filelist:
            input_df = process_input_file(file)
            process_sentence_df(input_df)
            numfiles += 1
    return numfiles
def process_hdfs_dir(dir_name):
    # Turn input files to Spark DataFrame
    print("START processing input HDFS directory...")
    data_df = spark.read.option("recursiveFileLookup",
"true").text(dir_name)
    data_df.show()
    print("[DEBUG] total lines in data_df = ", data_df.count())
    # rename first column 'text' for sparknlp
    output_df = data_df.withColumnRenamed("value", "text").dropna()
    print("[DEBUG] output_df looks like: \n")
    output_df.show(40, False)
    print("[DEBUG] HDFS dir resulting data_df schema: \n")
    output_df.printSchema()
    process_sentence_df(output_df)
    print("Processed HDFS directory: ", dir_name)
    returnif __name__ == '__main__':
    try:
        if len(argv) == 2:
            print("Start processing input...\n")
    except:
        print("[ERROR] Please enter input text file or path to
process!\n")
        exit(1)
    # This is for local file, not hdfs:
    numfiles = process_local_dir_or_file(str(argv[1]))
    # For HDFS single file & directory:
    input_df = process_input_file(str(argv[1]))
    print("Obtained input_df.")
    process_sentence_df(input_df)
    print("Processed input_df")
```

```
    numfiles += 1
    # For HDFS directory of subdirectories of files:
    input_parse_list = str(argv[1]).split('/')
    print(input_parse_list)
    if input_parse_list[-2:-1] == ['Transcripts']:
        print("Start processing HDFS directory: ", str(argv[1]))
        process_hdfs_dir(str(argv[1]))
    print(f"[OK!] All done. Number of files processed = {numfiles}")
```

2番目のスクリプトは’kers_spark_horovod_Rossmann _ estimator.pyです

```
# Copyright 2022 NetApp, Inc.
# Authored by Rick Huang
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#
==========================================================================
====
# The below code was modified from: https://www.kaggle.com/c/rossmann-
store-sales
import argparse
import datetime
import os
import sys
from distutils.version import LooseVersion
import pyspark.sql.types as T
import pyspark.sql.functions as F
from pyspark import SparkConf, Row
from pyspark.sql import SparkSession
import tensorflow as tf
import tensorflow.keras.backend as K
from tensorflow.keras.layers import Input, Embedding, Concatenate, Dense,
Flatten, Reshape, BatchNormalization, Dropout
import horovod.spark.keras as hvd
from horovod.spark.common.backend import SparkBackend
```

```
from horovod.spark.common.store import Store
from horovod.tensorflow.keras.callbacks import BestModelCheckpoint
parser = argparse.ArgumentParser(description='Horovod Keras Spark Rossmann
Estimator Example',

formatter_class=argparse.ArgumentDefaultsHelpFormatter)
parser.add_argument('--master',
                    help='spark cluster to use for training. If set to
None, uses current default cluster. Cluster'
                         'should be set up to provide a Spark task per
multiple CPU cores, or per GPU, e.g. by'
                         'supplying `-c <NUM_GPUS>` in Spark Standalone
mode')
parser.add_argument('--num-proc', type=int,
                    help='number of worker processes for training,
default: `spark.default.parallelism`')
parser.add_argument('--learning_rate', type=float, default=0.0001,
                    help='initial learning rate')
parser.add_argument('--batch-size', type=int, default=100,
                    help='batch size')
parser.add_argument('--epochs', type=int, default=100,
                    help='number of epochs to train')
parser.add_argument('--sample-rate', type=float,
                    help='desired sampling rate. Useful to set to low
number (e.g. 0.01) to make sure that '
                         'end-to-end process works')
parser.add_argument('--data-dir', default='file://' + os.getcwd(),
                    help='location of data on local filesystem (prefixed
with file://) or on HDFS')
parser.add_argument('--local-submission-csv', default='submission.csv',
                    help='output submission predictions CSV')
parser.add_argument('--local-checkpoint-file', default='checkpoint',
                    help='model checkpoint')
parser.add_argument('--work-dir', default='/tmp',
                    help='temporary working directory to write
intermediate files (prefix with hdfs:// to use HDFS)')
if __name__ == '__main__':
    args = parser.parse_args()
    # ================ #
    # DATA PREPARATION #
    # ================ #
    print('================')
    print('Data preparation')
    print('================')
    # Create Spark session for data preparation.
    conf = SparkConf() \
```

```
        .setAppName('Keras Spark Rossmann Estimator Example') \
        .set('spark.sql.shuffle.partitions', '480') \
        .set("spark.executor.cores", "1") \
        .set('spark.executor.memory', '5gb') \
        .set('spark.executor.memoryOverhead','1000')\
        .set('spark.driver.memoryOverhead','1000')
    if args.master:
        conf.setMaster(args.master)
    elif args.num_proc:
        conf.setMaster('local[{}]'.format(args.num_proc))
    spark = SparkSession.builder.config(conf=conf).getOrCreate()
    train_csv = spark.read.csv('%s/train.csv' % args.data_dir,
header=True)
    test_csv = spark.read.csv('%s/test.csv' % args.data_dir, header=True)
    store_csv = spark.read.csv('%s/store.csv' % args.data_dir,
header=True)
    store_states_csv = spark.read.csv('%s/store_states.csv' %
args.data_dir, header=True)
    state_names_csv = spark.read.csv('%s/state_names.csv' % args.data_dir,
header=True)
    google_trend_csv = spark.read.csv('%s/googletrend.csv' %
args.data_dir, header=True)
    weather_csv = spark.read.csv('%s/weather.csv' % args.data_dir,
header=True)
    def expand_date(df):
        df = df.withColumn('Date', df.Date.cast(T.DateType()))
        return df \
            .withColumn('Year', F.year(df.Date)) \
            .withColumn('Month', F.month(df.Date)) \
            .withColumn('Week', F.weekofyear(df.Date)) \
            .withColumn('Day', F.dayofmonth(df.Date))
    def prepare_google_trend():
        # Extract week start date and state.
        google_trend_all = google_trend_csv \
            .withColumn('Date', F.regexp_extract(google_trend_csv.week,
'(.*?) -', 1)) \
            .withColumn('State', F.regexp_extract(google_trend_csv.file,
'Rossmann_DE_(.*)', 1))
        # Map state NI -> HB,NI to align with other data sources.
        google_trend_all = google_trend_all \
            .withColumn('State', F.when(google_trend_all.State == 'NI',
'HB,NI').otherwise(google_trend_all.State))
        # Expand dates.
        return expand_date(google_trend_all)
    def add_elapsed(df, cols):
        def add_elapsed_column(col, asc):
```

```
            def fn(rows):
                last_store, last_date = None, None
                for r in rows:
                    if last_store != r.Store:
                        last_store = r.Store
                        last_date = r.Date
                    if r[col]:
                        last_date = r.Date
                    fields = r.asDict().copy()
                    fields[('After' if asc else 'Before') + col] = (r.Date
- last_date).days
                    yield Row(**fields)
            return fn
        df = df.repartition(df.Store)
        for asc in [False, True]:
            sort_col = df.Date.asc() if asc else df.Date.desc()
            rdd = df.sortWithinPartitions(df.Store.asc(), sort_col).rdd
            for col in cols:
                rdd = rdd.mapPartitions(add_elapsed_column(col, asc))
            df = rdd.toDF()
        return df
    def prepare_df(df):
        num_rows = df.count()
        # Expand dates.
        df = expand_date(df)
        df = df \
            .withColumn('Open', df.Open != '0') \
            .withColumn('Promo', df.Promo != '0') \
            .withColumn('StateHoliday', df.StateHoliday != '0') \
            .withColumn('SchoolHoliday', df.SchoolHoliday != '0')
        # Merge in store information.
        store = store_csv.join(store_states_csv, 'Store')
        df = df.join(store, 'Store')
        # Merge in Google Trend information.
        google_trend_all = prepare_google_trend()
        df = df.join(google_trend_all, ['State', 'Year',
'Week']).select(df['*'], google_trend_all.trend)
        # Merge in Google Trend for whole Germany.
        google_trend_de = google_trend_all[google_trend_all.file ==
'Rossmann_DE'].withColumnRenamed('trend', 'trend_de')
        df = df.join(google_trend_de, ['Year', 'Week']).select(df['*'],
google_trend_de.trend_de)
        # Merge in weather.
        weather = weather_csv.join(state_names_csv, weather_csv.file ==
state_names_csv.StateName)
        df = df.join(weather, ['State', 'Date'])
```

```
        # Fix null values.
        df = df \
            .withColumn('CompetitionOpenSinceYear',
F.coalesce(df.CompetitionOpenSinceYear, F.lit(1900))) \
            .withColumn('CompetitionOpenSinceMonth',
F.coalesce(df.CompetitionOpenSinceMonth, F.lit(1))) \
            .withColumn('Promo2SinceYear', F.coalesce(df.Promo2SinceYear,
F.lit(1900))) \
            .withColumn('Promo2SinceWeek', F.coalesce(df.Promo2SinceWeek,
F.lit(1)))
        # Days & months competition was open, cap to 2 years.
        df = df.withColumn('CompetitionOpenSince',
                           F.to_date(F.format_string('%s-%s-15',
df.CompetitionOpenSinceYear,

df.CompetitionOpenSinceMonth)))
        df = df.withColumn('CompetitionDaysOpen',
                           F.when(df.CompetitionOpenSinceYear > 1900,
                                  F.greatest(F.lit(0), F.least(F.lit(360 *
2), F.datediff(df.Date, df.CompetitionOpenSince))))
                           .otherwise(0))
        df = df.withColumn('CompetitionMonthsOpen',
(df.CompetitionDaysOpen / 30).cast(T.IntegerType()))
        # Days & weeks of promotion, cap to 25 weeks.
        df = df.withColumn('Promo2Since',
                           F.expr('date_add(format_string("%s-01-01",
Promo2SinceYear), (cast(Promo2SinceWeek as int) - 1) * 7)'))
        df = df.withColumn('Promo2Days',
                           F.when(df.Promo2SinceYear > 1900,
                                  F.greatest(F.lit(0), F.least(F.lit(25 *
7), F.datediff(df.Date, df.Promo2Since))))
                           .otherwise(0))
        df = df.withColumn('Promo2Weeks', (df.Promo2Days /
7).cast(T.IntegerType()))
        # Check that we did not lose any rows through inner joins.
        assert num_rows == df.count(), 'lost rows in joins'
        return df
    def build_vocabulary(df, cols):
        vocab = {}
        for col in cols:
            values = [r[0] for r in df.select(col).distinct().collect()]
            col_type = type([x for x in values if x is not None][0])
            default_value = col_type()
            vocab[col] = sorted(values, key=lambda x: x or default_value)
        return vocab
    def cast_columns(df, cols):
```

```
        for col in cols:
            df = df.withColumn(col,
F.coalesce(df[col].cast(T.FloatType()), F.lit(0.0)))
        return df
    def lookup_columns(df, vocab):
        def lookup(mapping):
            def fn(v):
                return mapping.index(v)
            return F.udf(fn, returnType=T.IntegerType())
        for col, mapping in vocab.items():
            df = df.withColumn(col, lookup(mapping)(df[col]))
        return df
    if args.sample_rate:
        train_csv = train_csv.sample(withReplacement=False,
fraction=args.sample_rate)
        test_csv = test_csv.sample(withReplacement=False,
fraction=args.sample_rate)
    # Prepare data frames from CSV files.
    train_df = prepare_df(train_csv).cache()
    test_df = prepare_df(test_csv).cache()
    # Add elapsed times from holidays & promos, the data spanning training
& test datasets.
    elapsed_cols = ['Promo', 'StateHoliday', 'SchoolHoliday']
    elapsed = add_elapsed(train_df.select('Date', 'Store', *elapsed_cols)
                          .unionAll(test_df.select('Date', 'Store',
*elapsed_cols)),
                          elapsed_cols)
    # Join with elapsed times.
    train_df = train_df \
        .join(elapsed, ['Date', 'Store']) \
        .select(train_df['*'], *[prefix + col for prefix in ['Before',
'After'] for col in elapsed_cols])
    test_df = test_df \
        .join(elapsed, ['Date', 'Store']) \
        .select(test_df['*'], *[prefix + col for prefix in ['Before',
'After'] for col in elapsed_cols])
    # Filter out zero sales.
    train_df = train_df.filter(train_df.Sales > 0)
    print('===================')
    print('Prepared data frame')
    print('===================')
    train_df.show()
    categorical_cols = [
        'Store', 'State', 'DayOfWeek', 'Year', 'Month', 'Day', 'Week',
'CompetitionMonthsOpen', 'Promo2Weeks', 'StoreType',
        'Assortment', 'PromoInterval', 'CompetitionOpenSinceYear',
```

```
'Promo2SinceYear', 'Events', 'Promo',
        'StateHoliday', 'SchoolHoliday'
    ]
    continuous_cols = [
        'CompetitionDistance', 'Max_TemperatureC', 'Mean_TemperatureC',
'Min_TemperatureC', 'Max_Humidity',
        'Mean_Humidity', 'Min_Humidity', 'Max_Wind_SpeedKm_h',
'Mean_Wind_SpeedKm_h', 'CloudCover', 'trend', 'trend_de',
        'BeforePromo', 'AfterPromo', 'AfterStateHoliday',
'BeforeStateHoliday', 'BeforeSchoolHoliday', 'AfterSchoolHoliday'
    ]
    all_cols = categorical_cols + continuous_cols
    # Select features.
    train_df = train_df.select(*(all_cols + ['Sales', 'Date'])).cache()
    test_df = test_df.select(*(all_cols + ['Id', 'Date'])).cache()
    # Build vocabulary of categorical columns.
    vocab = build_vocabulary(train_df.select(*categorical_cols)

.unionAll(test_df.select(*categorical_cols)).cache(),
                             categorical_cols)
    # Cast continuous columns to float & lookup categorical columns.
    train_df = cast_columns(train_df, continuous_cols + ['Sales'])
    train_df = lookup_columns(train_df, vocab)
    test_df = cast_columns(test_df, continuous_cols)
    test_df = lookup_columns(test_df, vocab)
    # Split into training & validation.
    # Test set is in 2015, use the same period in 2014 from the training
set as a validation set.
    test_min_date = test_df.agg(F.min(test_df.Date)).collect()[0][0]
    test_max_date = test_df.agg(F.max(test_df.Date)).collect()[0][0]
    one_year = datetime.timedelta(365)
    train_df = train_df.withColumn('Validation',
                                   (train_df.Date > test_min_date -
one_year) & (train_df.Date <= test_max_date - one_year))
    # Determine max Sales number.
    max_sales = train_df.agg(F.max(train_df.Sales)).collect()[0][0]
    # Convert Sales to log domain
    train_df = train_df.withColumn('Sales', F.log(train_df.Sales))
    print('=====================================')
    print('Data frame with transformed columns')
    print('=====================================')
    train_df.show()
    print('================')
    print('Data frame sizes')
    print('================')
    train_rows = train_df.filter(~train_df.Validation).count()
```

```
    val_rows = train_df.filter(train_df.Validation).count()
    test_rows = test_df.count()
    print('Training: %d' % train_rows)
    print('Validation: %d' % val_rows)
    print('Test: %d' % test_rows)
    # ============== #
    # MODEL TRAINING #
    # ============== #
    print('==============')
    print('Model training')
    print('==============')
    def exp_rmspe(y_true, y_pred):
        """Competition evaluation metric, expects logarithic inputs."""
        pct = tf.square((tf.exp(y_true) - tf.exp(y_pred)) /
tf.exp(y_true))
        # Compute mean excluding stores with zero denominator.
        x = tf.reduce_sum(tf.where(y_true > 0.001, pct,
tf.zeros_like(pct)))
        y = tf.reduce_sum(tf.where(y_true > 0.001, tf.ones_like(pct),
tf.zeros_like(pct)))
        return tf.sqrt(x / y)
    def act_sigmoid_scaled(x):
        """Sigmoid scaled to logarithm of maximum sales scaled by 20%."""
        return tf.nn.sigmoid(x) * tf.math.log(max_sales) * 1.2
    CUSTOM_OBJECTS = {'exp_rmspe': exp_rmspe,
                      'act_sigmoid_scaled': act_sigmoid_scaled}
    # Disable GPUs when building the model to prevent memory leaks
    if LooseVersion(tf.__version__) >= LooseVersion('2.0.0'):
        # See https://github.com/tensorflow/tensorflow/issues/33168
        os.environ['CUDA_VISIBLE_DEVICES'] = '-1'
    else:

K.set_session(tf.Session(config=tf.ConfigProto(device_count={'GPU': 0})))
    # Build the model.
    inputs = {col: Input(shape=(1,), name=col) for col in all_cols}
    embeddings = [Embedding(len(vocab[col]), 10, input_length=1,
name='emb_' + col)(inputs[col])
                  for col in categorical_cols]
    continuous_bn = Concatenate()([Reshape((1, 1), name='reshape_' +
col)(inputs[col])
                                   for col in continuous_cols])
    continuous_bn = BatchNormalization()(continuous_bn)
    x = Concatenate()(embeddings + [continuous_bn])
    x = Flatten()(x)
    x = Dense(1000, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.00005))(x)
```

```
    x = Dense(1000, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.00005))(x)
    x = Dense(1000, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.00005))(x)
    x = Dense(500, activation='relu',
kernel_regularizer=tf.keras.regularizers.l2(0.00005))(x)
    x = Dropout(0.5)(x)
    output = Dense(1, activation=act_sigmoid_scaled)(x)
    model = tf.keras.Model([inputs[f] for f in all_cols], output)
    model.summary()
    opt = tf.keras.optimizers.Adam(lr=args.learning_rate, epsilon=1e-3)
    # Checkpoint callback to specify options for the returned Keras model
    ckpt_callback = BestModelCheckpoint(monitor='val_loss', mode='auto',
save_freq='epoch')
    # Horovod: run training.
    store = Store.create(args.work_dir)
    backend = SparkBackend(num_proc=args.num_proc,
                           stdout=sys.stdout, stderr=sys.stderr,
                           prefix_output_with_timestamp=True)
    keras_estimator = hvd.KerasEstimator(backend=backend,
                                         store=store,
                                         model=model,
                                         optimizer=opt,
                                         loss='mae',
                                         metrics=[exp_rmspe],
                                         custom_objects=CUSTOM_OBJECTS,
                                         feature_cols=all_cols,
                                         label_cols=['Sales'],
                                         validation='Validation',
                                         batch_size=args.batch_size,
                                         epochs=args.epochs,
                                         verbose=2,

checkpoint_callback=ckpt_callback)
    keras_model =
keras_estimator.fit(train_df).setOutputCols(['Sales_output'])
    history = keras_model.getHistory()
    best_val_rmspe = min(history['val_exp_rmspe'])
    print('Best RMSPE: %f' % best_val_rmspe)
    # Save the trained model.
    keras_model.save(args.local_checkpoint_file)
    print('Written checkpoint to %s' % args.local_checkpoint_file)
    # ================ #
    # FINAL PREDICTION #
    # ================ #
    print('================')
```

```
    print('Final prediction')
    print('================')
    pred_df=keras_model.transform(test_df)
    pred_df.printSchema()
    pred_df.show(5)
    # Convert from log domain to real Sales numbers
    pred_df=pred_df.withColumn('Sales_pred', F.exp(pred_df.Sales_output))
    submission_df = pred_df.select(pred_df.Id.cast(T.IntegerType()),
pred_df.Sales_pred).toPandas()
    submission_df.sort_values(by=['Id']).to_csv(args.local_submission_csv,
index=False)
    print('Saved predictions to %s' % args.local_submission_csv)
    spark.stop()
```

3番目のスクリプトは'run_classification_Crito_spark.pyです

```
import tempfile, string, random, os, uuid
import argparse, datetime, sys, shutil
import csv
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.callbacks import EarlyStopping
from pyspark import SparkContext
from pyspark.sql import SparkSession, SQLContext, Row, DataFrame
from pyspark.mllib import linalg as mllib_linalg
from pyspark.mllib.linalg import SparseVector as mllibSparseVector
from pyspark.mllib.linalg import VectorUDT as mllibVectorUDT
from pyspark.mllib.linalg import Vector as mllibVector, Vectors as
mllibVectors
from pyspark.mllib.regression import LabeledPoint
from pyspark.mllib.classification import LogisticRegressionWithSGD
from pyspark.ml import linalg as ml_linalg
from pyspark.ml.linalg import VectorUDT as mlVectorUDT
from pyspark.ml.linalg import SparseVector as mlSparseVector
from pyspark.ml.linalg import Vector as mlVector, Vectors as mlVectors
from pyspark.ml.classification import LogisticRegression
from pyspark.ml.feature import OneHotEncoder
from math import log
from math import exp  # exp(-t) = e^-t
from operator import add
from pyspark.sql.functions import udf, split, lit
from pyspark.sql.functions import size, sum as sqlsum
import pyspark.sql.functions as F
import pyspark.sql.types as T
from pyspark.sql.types import ArrayType, StructType, StructField,
```

```
LongType, StringType, IntegerType, FloatType
from pyspark.sql.functions import explode, col, log, when
from collections import defaultdict
import pandas as pd
import pyspark.pandas as ps
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from deepctr.models import DeepFM
from deepctr.feature_column import SparseFeat, DenseFeat,
get_feature_names
spark = SparkSession.builder \
    .master("yarn") \
    .appName("deep_ctr_classification") \
    .config("spark.jars.packages", "io.github.ravwojdyla:spark-schema-
utils_2.12:0.1.0") \
    .config("spark.executor.cores", "1") \
    .config('spark.executor.memory', '5gb') \
    .config('spark.executor.memoryOverhead', '1500') \
    .config('spark.driver.memoryOverhead', '1500') \
    .config("spark.sql.shuffle.partitions", "480") \
    .config("spark.sql.execution.arrow.enabled", "true") \
    .config("spark.driver.maxResultSize", "50gb") \
    .getOrCreate()
# spark.conf.set("spark.sql.execution.arrow.enabled", "true") # deprecated
print("Apache Spark version:")
print(spark.version)
sc = spark.sparkContext
sqlContext = SQLContext(sc)
parser = argparse.ArgumentParser(description='Spark DCN CTR Prediction
Example',

formatter_class=argparse.ArgumentDefaultsHelpFormatter)
parser.add_argument('--data-dir', default='file://' + os.getcwd(),
                    help='location of data on local filesystem (prefixed
with file://) or on HDFS')
def process_input_file(file_name, sparse_feat, dense_feat):
    # Need this preprocessing to turn Criteo raw file into CSV:
    print("START processing input file...")
    # only convert the file ONCE
    # sample = open(file_name)
    # sample = '\n'.join([str(x.replace('\n', '').replace('\t', ',')) for
x in sample])
    # # Add header in data file and save as CSV
    # header = ','.join(str(x) for x in (['label'] + dense_feat +
sparse_feat))
```

```
    # with open('/sparkdemo/tr-4570-data/ctr_train.csv', mode='w',
encoding="utf-8") as f:
    #     f.write(header + '\n' + sample)
    #     f.close()
    # print("Raw training file processed and saved as CSV: ", f.name)
    raw_df = sqlContext.read.option("header", True).csv(file_name)
    raw_df.show(5, False)
    raw_df.printSchema()
    # convert columns I1 to I13 from string to integers
    conv_df = raw_df.select(col('label').cast("double"),
                            *(col(i).cast("float").alias(i) for i in
raw_df.columns if i in dense_feat),
                            *(col(c) for c in raw_df.columns if c in
sparse_feat))
    print("Schema of raw_df with integer columns type changed:")
    conv_df.printSchema()
    # result_pdf = conv_df.select("*").toPandas()
    tmp_df = conv_df.na.fill(0, dense_feat)
    result_df = tmp_df.na.fill('-1', sparse_feat)
    result_df.show()
    return result_df
if __name__ == "__main__":
    args = parser.parse_args()
    # Pandas read CSV
    # data = pd.read_csv('%s/criteo_sample.txt' % args.data_dir)
    # print("Obtained Pandas df.")
    dense_features = ['I' + str(i) for i in range(1, 14)]
    sparse_features = ['C' + str(i) for i in range(1, 27)]
    # Spark read CSV
    # process_input_file('%s/train.txt' % args.data_dir, sparse_features,
dense_features) # run only ONCE
    spark_df = process_input_file('%s/data.txt' % args.data_dir,
sparse_features, dense_features) # sample data
    # spark_df = process_input_file('%s/ctr_train.csv' % args.data_dir,
sparse_features, dense_features)
    print("Obtained Spark df and filled in missing features.")
    data = spark_df
    # Pandas
    #data[sparse_features] = data[sparse_features].fillna('-1', )
    #data[dense_features] = data[dense_features].fillna(0, )
    target = ['label']
    label_npa = data.select("label").toPandas().to_numpy()
    print("label numPy array has length = ", len(label_npa)) # 45,840,617
w/ 11GB dataset
    label_npa.ravel()
    label_npa.reshape(len(label_npa), )
```

```
    # 1.Label Encoding for sparse features,and do simple Transformation
for dense features
    print("Before LabelEncoder():")
    data.printSchema()  # label: float (nullable = true)
    for feat in sparse_features:
        lbe = LabelEncoder()
        tmp_pdf = data.select(feat).toPandas().to_numpy()
        tmp_ndarray = lbe.fit_transform(tmp_pdf)
        print("After LabelEncoder(), tmp_ndarray[0] =", tmp_ndarray[0])
        # print("Data tmp PDF after lbe transformation, the output ndarray
has length = ", len(tmp_ndarray)) # 45,840,617 for 11GB dataset
        tmp_ndarray.ravel()
        tmp_ndarray.reshape(len(tmp_ndarray), )
        out_ndarray = np.column_stack([label_npa, tmp_ndarray])
        pdf = pd.DataFrame(out_ndarray, columns=['label', feat])
        s_df = spark.createDataFrame(pdf)
        s_df.printSchema() # label: double (nullable = true)
        print("Before joining data df with s_df, s_df example rows:")
        s_df.show(1, False)
        data = data.drop(feat).join(s_df, 'label').drop('label')
        print("After LabelEncoder(), data df example rows:")
        data.show(1, False)
        print("Finished processing sparse_features: ", feat)
    print("Data DF after label encoding: ")
    data.show()
    data.printSchema()
    mms = MinMaxScaler(feature_range=(0, 1))
    # data[dense_features] = mms.fit_transform(data[dense_features]) # for
Pandas df
    tmp_pdf = data.select(dense_features).toPandas().to_numpy()
    tmp_ndarray = mms.fit_transform(tmp_pdf)
    tmp_ndarray.ravel()
    tmp_ndarray.reshape(len(tmp_ndarray), len(tmp_ndarray[0]))
    out_ndarray = np.column_stack([label_npa, tmp_ndarray])
    pdf = pd.DataFrame(out_ndarray, columns=['label'] + dense_features)
    s_df = spark.createDataFrame(pdf)
    s_df.printSchema()
    data.drop(*dense_features).join(s_df, 'label').drop('label')
    print("Finished processing dense_features: ", dense_features)
    print("Data DF after MinMaxScaler: ")
    data.show()

    # 2.count #unique features for each sparse field,and record dense
feature field name
    fixlen_feature_columns = [SparseFeat(feat,
vocabulary_size=data.select(feat).distinct().count() + 1, embedding_dim=4)
```

```
                                for i, feat in enumerate(sparse_features)] +
\
                               [DenseFeat(feat, 1, ) for feat in
dense_features]
    dnn_feature_columns = fixlen_feature_columns
    linear_feature_columns = fixlen_feature_columns
    feature_names = get_feature_names(linear_feature_columns +
dnn_feature_columns)
    # 3.generate input data for model
    # train, test = train_test_split(data.toPandas(), test_size=0.2,
random_state=2020) # Pandas; might hang for 11GB data
    train, test = data.randomSplit(weights=[0.8, 0.2], seed=200)
    print("Training dataset size = ", train.count())
    print("Testing dataset size = ", test.count())
    # Pandas:
    # train_model_input = {name: train[name] for name in feature_names}
    # test_model_input = {name: test[name] for name in feature_names}
    # Spark DF:
    train_model_input = {}
    test_model_input = {}
    for name in feature_names:
        if name.startswith('I'):
            tr_pdf = train.select(name).toPandas()
            train_model_input[name] = pd.to_numeric(tr_pdf[name])
            ts_pdf = test.select(name).toPandas()
            test_model_input[name] = pd.to_numeric(ts_pdf[name])
    # 4.Define Model,train,predict and evaluate
    model = DeepFM(linear_feature_columns, dnn_feature_columns,
task='binary')
    model.compile("adam", "binary_crossentropy",
                  metrics=['binary_crossentropy'], )
    lb_pdf = train.select(target).toPandas()
    history = model.fit(train_model_input,
pd.to_numeric(lb_pdf['label']).values,
                        batch_size=256, epochs=10, verbose=2,
validation_split=0.2, )
    pred_ans = model.predict(test_model_input, batch_size=256)
    print("test LogLoss",
round(log_loss(pd.to_numeric(test.select(target).toPandas()).values,
pred_ans), 4))
    print("test AUC",
round(roc_auc_score(pd.to_numeric(test.select(target).toPandas()).values,
pred_ans), 4))
```

まとめ

本ドキュメントでは、Apache Sparkのアーキテクチャ、お客様のユースケース、ネットアップストレージポートフォリオについて、ビッグデータ、最新の分析、AI、ML、DLに関連する情報をご紹介します。業界標準のベンチマークツールとお客様からの要望に基づくネットアップのパフォーマンス検証テストでは、NetApp Sparkソリューションは、ネイティブのHadoopシステムと比較して優れたパフォーマンスを実証しました。このレポートで紹介したお客様のユースケースとパフォーマンス結果を組み合わせることで、導入環境に適したSpark解決策 を選択できます。

追加情報の参照先

このTRでは次の資料を参照しています。

- Apache Sparkのアーキテクチャとコンポーネント

  "http://spark.apache.org/docs/latest/cluster-overview.html"

- Apache Sparkのユースケース

  "https://www.qubole.com/blog/big-data/apache-spark-use-cases/"

- Apacheの課題

  "http://www.infoworld.com/article/2897287/big-data/5-reasons-to-turn-to-spark-for-big-data-analytics.html"

- SparkのNLPです

  "https://www.johnsnowlabs.com/spark-nlp/"

- BERT

  "https://arxiv.org/abs/1810.04805"

- 広告クリック予測のためのディープおよびクロスネットワーク

  "https://arxiv.org/abs/1708.05123"

- FlexGroup

  "http://www.netapp.com/us/media/tr-4557.pdf"

- ストリーミングETL

  "https://www.infoq.com/articles/apache-spark-streaming"

- NetApp EシリーズHadoop向けソリューション

  "https://www.netapp.com/media/16420-tr-3969.pdf"

- 顧客コミュニケーションからネットアップAIとの感情分析

"https://docs.netapp.com/us-en/netapp-solutions/pdfs/sidebar/Sentiment_analysis_with_NetApp_AI.pdf"

- ネットアップの最新データ分析ソリューション

  "https://docs.netapp.com/us-en/netapp-solutions/data-analytics/index.html"

- SnapMirror

  "https://docs.netapp.com/us-en/ontap/data-protection/snapmirror-replication-concept.html"

- XCP

  https://mysupport.netapp.com/documentation/docweb/index.html?productID=63942&language=en-US

- BlueXPのコピーと同期

  "https://cloud.netapp.com/cloud-sync-service"

- DataOpsツールキット

  "https://github.com/NetApp/netapp-dataops-toolkit"

著作権に関する情報

Copyright © 2024 NetApp, Inc. All Rights Reserved. Printed in the U.S.このドキュメントは著作権によって保護されています。著作権所有者の書面による事前承諾がある場合を除き、画像媒体、電子媒体、および写真複写、記録媒体、テープ媒体、電子検索システムへの組み込みを含む機械媒体など、いかなる形式および方法による複製も禁止します。

ネットアップの著作物から派生したソフトウェアは、次に示す使用許諾条項および免責条項の対象となります。

このソフトウェアは、ネットアップによって「現状のまま」提供されています。ネットアップは明示的な保証、または商品性および特定目的に対する適合性の暗示的保証を含み、かつこれに限定されないいかなる暗示的な保証も行いません。ネットアップは、代替品または代替サービスの調達、使用不能、データ損失、利益損失、業務中断を含み、かつこれに限定されない、このソフトウェアの使用により生じたすべての直接的損害、間接的損害、偶発的損害、特別損害、懲罰的損害、必然的損害の発生に対して、損失の発生の可能性が通知されていたとしても、その発生理由、根拠とする責任論、契約の有無、厳格責任、不法行為（過失またはそうでない場合を含む）にかかわらず、一切の責任を負いません。

ネットアップは、ここに記載されているすべての製品に対する変更を随時、予告なく行う権利を保有します。ネットアップによる明示的な書面による合意がある場合を除き、ここに記載されている製品の使用により生じる責任および義務に対して、ネットアップは責任を負いません。この製品の使用または購入は、ネットアップの特許権、商標権、または他の知的所有権に基づくライセンスの供与とはみなされません。

このマニュアルに記載されている製品は、1つ以上の米国特許、その他の国の特許、および出願中の特許によって保護されている場合があります。

権利の制限について：政府による使用、複製、開示は、DFARS 252.227-7013（2014年2月）およびFAR 5252.227-19（2007年12月）のRights in Technical Data -Noncommercial Items（技術データ - 非商用品目に関する諸権利）条項の(b)(3)項、に規定された制限が適用されます。

本書に含まれるデータは商用製品および / または商用サービス（FAR 2.101の定義に基づく）に関係し、データの所有権はNetApp, Inc.にあります。本契約に基づき提供されるすべてのネットアップの技術データおよびコンピュータ ソフトウェアは、商用目的であり、私費のみで開発されたものです。米国政府は本データに対し、非独占的かつ移転およびサブライセンス不可で、全世界を対象とする取り消し不能の制限付き使用権を有し、本データの提供の根拠となった米国政府契約に関連し、当該契約の裏付けとする場合にのみ本データを使用できます。前述の場合を除き、NetApp, Inc.の書面による許可を事前に得ることなく、本データを使用、開示、転載、改変するほか、上演または展示することはできません。国防総省にかかる米国政府のデータ使用権については、DFARS 252.227-7015(b)項（2014年2月）で定められた権利のみが認められます。

商標に関する情報

NetApp、NetAppのロゴ、http://www.netapp.com/TMに記載されているマークは、NetApp, Inc.の商標です。その他の会社名と製品名は、それを所有する各社の商標である場合があります。